CTU Training Solutions, Cape Town Campus

SPM622 Software Engineering 2: Project & Dissertation

TrackWIL

A web application for Work-Integrated Learning logbooks with account-free mentor verification and progress monitoring

Dissertation draft (Chapters 1 to 5 and an outline of Chapter 6)

Group 3: Willem (Gedion) Basson, Moegammad Green, Dylan Shepard, Sihle Kayise

Supervisor: Ms Amenda Raganya

Week 4 submission: 23 October 2026

Team Details

Table 1: Team details

Item	Details
Project title	TrackWIL: Work-Integrated Learning Logbook Tracking
Group	Group 3, Cape Town Campus
Group members	Willem (Gedion) Basson (Group Lead), Moegammad Green (Development Lead), Dylan Shepard (Testing / QA), Sihle Kayise (Group Member)
Module	SPM622 (Software Engineering 2)
Supervisor	Ms Amenda Raganya
Stage	Week 4: Build, Test and Presentation
Submission date	23 October 2026
GitHub repository	https://github.com/Noobey32/TrackWIL
Walkthrough video	[Insert the link to the 3-5 minute screen recording]

Acknowledgements

We thank our supervisor, Ms Amenda Raganya, for her guidance and feedback on each weekly submission, which shaped the scope and quality of this project. We also thank CTU Training Solutions for the opportunity to apply our software engineering studies to a real problem in Work-Integrated Learning.

Declaration

We, the members of Group 3, declare that this dissertation and the TrackWIL application are the work of our group, that every source used has been acknowledged and referenced in the Harvard style, and that each member's contribution is recorded accurately in the Individual Contribution Forms and the Git history.

Table 2: Signatures

Member	Signature	Date
Willem (Gedion) Basson		
Moegammad Green		
Dylan Shepard		
Sihle Kayise		

Abstract

Work-Integrated Learning (WIL) students must record their daily workplace activities and have them signed by a workplace mentor. When this is done with paper, PDF forms and email, documents are lost, signatures are delayed and coordinators cannot see which students are falling behind. This project aimed to design, develop and evaluate TrackWIL, a web application in which students record daily logbook entries with evidence, workplace mentors verify each entry through a secure one-time link without needing an account, and coordinators and lecturers monitor every student's progress on one dashboard. A hybrid Agile (Scrum) approach with one-week sprints was used. The system was built with Next.js and TypeScript and an SQLite relational database, and its design is documented with use case, class, activity and sequence diagrams and an entity-relationship diagram. TrackWIL was tested with 55 test cases, combining manual tests, 36 automated system tests and browser checks: 54 passed and one usability test is waiting for ethical permission. Testing found and fixed a critical session-cookie vulnerability, an expired-link problem and a mobile layout defect. All fourteen functional requirements and six of the seven non-functional requirements were met, and pages responded in under 10 milliseconds on the test computer. The results show that account-free, link-based verification combined with automatic status monitoring can replace the manual logbook process; the next step is to measure its usability with real students, mentors and staff.

Table of Contents

Team Details 2
Acknowledgements 3
Declaration 4
Abstract 5
List of Abbreviations 10
Key Terms 11
CHAPTER 1 - INTRODUCTION 12
1.1 Background of the Study 12
1.2 Problem Statement 12
1.3 Research Aim 13
1.4 Research Objectives 13
1.5 Research Questions 13
1.6 Proposed Software Solution 13
1.7 Scope 14
1.8 Significance of the Study 14
1.9 Chapter Summary 15
CHAPTER 2 - LITERATURE REVIEW 16
2.1 Introduction 16
2.2 Existing Research 16
2.3 Existing or Similar Systems 17
2.4 Technology Review 18
2.5 Research Gap 19
2.6 Conceptual / Technical Direction 19
2.7 Chapter Summary 20
CHAPTER 3 - REQUIREMENTS & METHODOLOGY 21
3.1 Introduction 21
3.2 Stakeholders and Users 21
3.3 Functional Requirements 22
3.4 Non-Functional Requirements 23
3.5 Requirements Gathering 24
3.6 Development Methodology 25
3.7 Use Cases / User Stories 25
3.8 System Architecture 26

3.9 Ethical, Privacy and Security Considerations 27
3.10 Chapter Summary 28
CHAPTER 4 - SYSTEM DESIGN & DEVELOPMENT 29
4.1 Introduction 29
4.2 System Architecture 29
4.3 UML Design 31
4.4 Database Design 35
4.5 User Interface Design 37
4.6 Development Environment 41
4.7 Implementation 42
4.8 Version Control 44
4.9 Chapter Summary 45
CHAPTER 5 - TESTING, RESULTS & EVALUATION 47
5.1 Introduction 47
5.2 Test Strategy 47
5.3 Test Cases and Results 47
5.4 Defects and Corrections 50
5.5 Performance / Usability Evaluation 51
5.6 Requirements Evaluation 51
5.7 User / Stakeholder Feedback 52
5.8 Chapter Summary 52
CHAPTER 6 - CONCLUSION & RECOMMENDATIONS (OUTLINE) 53
6.1 Introduction 53
6.2 Summary of Findings 53
6.3 Achievement of Objectives 53
6.4 Answers to Research Questions 53
6.5 Limitations 54
6.6 Recommendations 54
6.7 Future Work 54
6.8 Final Conclusion 54
References 55
Appendices 57
Appendix A: Source Code and How to Run TrackWIL 57
Appendix B: Supporting Documents 57
Appendix C: Mobile Test Evidence 58

List of Figures

Figure 1: TrackWIL system architecture 29
Figure 2: Use case diagram 32
Figure 3: Class diagram of the main types and modules 33
Figure 4: Activity diagram of the logbook and verification workflow 34
Figure 5: Sequence diagram of mentor verification 35
Figure 6: Entity-relationship diagram (SQLite) 36
Figure 7: Sign-in page with an error for a wrong password 37
Figure 8: Student logbook showing verified hours and entry statuses 38
Figure 9: New logbook entry form 38
Figure 10: Entry details with the mentor verification link 39
Figure 11: Mentor verification page (no sign-in required) 40
Figure 12: Coordinator dashboard with On track, Awaiting verification and Overdue statuses 41
Figure 13: Collaborators on the GitHub repository 45
Figure 14: Manual test TC-011: an invalid sign-in is refused with a clear message (Sihle Kayise, 22 October 2026) 49
Figure 15: Manual test TC-009: coordinator dashboard during manual testing (Sihle Kayise, 22 October 2026) 50
Figure 16: Mentor verification page at 375 px in Chrome after the DEF-03 fix (TC-145) 58
Figure 17: Coordinator dashboard at 375 px in Edge after the DEF-03 fix: the table scrolls inside its box (TC-145) 59

List of Tables

Table 1: Team details 2
Table 2: Signatures 4
Table 3: Abbreviations 10
Table 4: Key terms 11
Table 5: Comparison of similar systems against TrackWIL's needs 17
Table 6: Stakeholders, their needs and how they use TrackWIL 21
Table 7: Functional requirements 22
Table 8: Requirements deferred to future versions 23
Table 9: Non-functional requirements 23
Table 10: Requirements-gathering methods and evidence 24
Table 11: Sprint plan 25
Table 12: User stories and acceptance criteria 25
Table 13: Main components and their responsibilities 30
Table 14: Technology stack and justification 31
Table 15: Usability decisions 41
Table 16: Development environment 41
Table 17: Source code structure (05_Source_Code/trackwil) 42
Table 18: Branches in the repository 44
Table 19: Commits and main contributions by member (at the time of writing) 45
Table 20: Summary of test results 47
Table 21: Defects found, fixes and retest results 50
Table 22: Measured response times (TC-144, 23 October 2026) 51
Table 23: Evaluation against the requirements 51
Table 24: Achievement of objectives (to be expanded) 53
Table 25: Demonstration accounts (synthetic data) 57
Table 26: Supporting documents in the repository 57

List of Abbreviations

Table 3: Abbreviations

Abbreviation	Meaning
CTU	CTU Training Solutions
DEF	Defect (Defect Log identifier)
ERD	Entity-relationship diagram
FR / NFR	Functional requirement / non-functional requirement
HMAC	Hash-based message authentication code
HTTP(S)	Hypertext Transfer Protocol (Secure)
JSON	JavaScript Object Notation
MVP	Minimum viable product
OWASP	Open Worldwide Application Security Project
POPIA	Protection of Personal Information Act 4 of 2013
PR	Pull request
QA	Quality assurance
RTM	Requirements Traceability Matrix
SQL	Structured Query Language
SRS	Software Requirements Specification
TC	Test case
UI	User interface
UML	Unified Modeling Language
WIL	Work-Integrated Learning

Key Terms

Table 4: Key terms

Term	Definition
Logbook entry	One day of WIL work recorded by a student: date, description, hours and the mentor's email address
Workplace mentor	The person at the host organisation who supervises the student and verifies their entries
Verification link (token)	A one-time web address containing a 48-character random token that lets a mentor approve or reject one entry without signing in; it expires after 7 days
Verified hours	The total hours of a student's entries that a mentor has approved
Overdue	A student with no verified entry in the last 14 days and no entry awaiting verification
Server Action	A Next.js function that runs on the server when a form is submitted
Transaction	A group of database changes that are saved together or not at all
Regression testing	Rerunning all tests after a change to make sure nothing that worked before has broken

CHAPTER 1 - INTRODUCTION

1.1 Background of the Study

Work-Integrated Learning (WIL) is an important part of many occupational and technical qualifications because it gives students an opportunity to gain practical workplace experience and apply the knowledge and skills they have learned during their studies. During a WIL placement, students are required to complete workplace activities, record the work they have done, complete daily documentation and provide evidence of their activities.

Currently, the management of WIL activities can involve students completing documents daily and sending them to their workplace mentors or supervisors for verification and signing. Students then must wait for the signed documents to be returned before they can continue with the submission or verification process. These documents may be exchanged through email or other communication methods, which can make it difficult for students and academic staff to keep track of all required documents.

When documents are stored in different locations or sent between students and mentors, there is also a risk that documents may be misplaced, accidentally deleted or not returned on time. Coordinators and lecturers may also have difficulty determining which students have completed their daily requirements and which students are still waiting for documentation to be completed.

TrackWIL is proposed as a digital solution to improve this process. It gives students one central place to record their daily WIL activities, attach evidence and request verification. Workplace mentors confirm the work through a secure link without needing an account, and coordinators and lecturers monitor student progress from a shared dashboard.

1.2 Problem Statement

The current WIL documentation process can be time-consuming because students are required to complete daily documents and send them to their workplace mentors for verification and signing. Students then must wait for the signed documents to be returned. When this process is managed through emails, separate files or other manual methods, documents can be misplaced, accidentally deleted or returned late.

This affects students, because they need to keep track of their daily documents and ensure that they are completed and signed by their mentors. Delays in receiving signed documents can also make it difficult for students to keep their WIL records up to date.

It also affects WIL coordinators, who need to monitor whether students are completing their required activities and receiving the necessary verification. Checking documents from multiple students can become difficult and time-consuming.

Lecturers are also affected because they need to identify which students are up to date and which students are falling behind. Without a centralised system, it can be difficult to obtain an accurate and current view of each student's WIL progress.

Workplace mentors are affected as well. They are busy staff at host organisations who are not employees of the institution, so a process that requires them to sign paper or PDF forms, or to create and remember another account, adds work and causes further delays.

1.3 Research Aim

The aim of this project is to design, develop and evaluate TrackWIL, a centralised web application that allows students to record their daily workplace activities and evidence, allows workplace mentors to verify those records through a secure link, and enables coordinators and lecturers to monitor student progress.

1.4 Research Objectives

1. Analyse the current WIL documentation and monitoring process to identify the challenges experienced by students, workplace mentors, coordinators and lecturers.

2. Design a WIL logbook system in which students submit daily entries and evidence, and workplace mentors verify each entry through a one-time link without needing an account.

3. Develop the TrackWIL web application with a student logbook, a mentor verification page and a monitoring dashboard shared by coordinators and lecturers.

4. Implement secure storage and status tracking so that every entry is recorded in a database with its status (Draft, Pending verification, Verified or Rejected) and timestamps.

5. Test and evaluate TrackWIL against its requirements to determine whether it makes WIL documentation, verification and progress monitoring more efficient.

1.5 Research Questions

1. What challenges are experienced by students, workplace mentors, coordinators and lecturers when managing WIL documentation and activities using the current process? (Objective 1)

2. How can a digital workflow, in which mentors verify entries through a one-time link, improve the submission and verification of students' daily WIL records? (Objective 2)

3. How can TrackWIL support each user group through role-specific views: a student logbook, a mentor verification page and a coordinator and lecturer dashboard? (Objective 3)

4. How can storing entries, evidence and verification status in a central database improve the management and monitoring of student WIL activities? (Objective 4)

5. To what extent does TrackWIL meet its requirements and improve the efficiency of WIL documentation, verification and progress monitoring? (Objective 5)

1.6 Proposed Software Solution

The proposed software solution is TrackWIL, a web-based application designed to improve the management of Work-Integrated Learning activities and documentation. Each user group sees a view that matches its role.

Students sign up with their CTU student email address and sign in to a personal logbook. They record each day's work (date, description and hours), name their workplace mentor's email address and may attach a PDF or image as evidence. When an entry is ready, the student submits it for verification. The system then creates a secure link that is valid for seven days and can be used only once, and the student sends this link to the mentor. The logbook shows the status of every entry and the verified hours completed against the required total.

Workplace mentors do not need an account. They open the link, review the entry and approve or reject it. If the link has already been used or has expired, the system tells the mentor and does not change the entry. A rejected entry can be corrected by the student and submitted again.

WIL coordinators and lecturers sign in to a shared monitoring dashboard. It lists every student with their verified hours, number of entries and number of entries awaiting verification, and labels each student as On track, Awaiting verification or Overdue (no verified work in the last 14 days). This allows staff to identify students who need follow-up.

1.7 Scope

The scope was set so that the core workflow could be built, tested and demonstrated within the four-week project period.

Included in the project

- Student sign-up with a CTU student email address and password rules, sign-in and sign-out.
- Role-based access for students, coordinators and lecturers.
- Daily logbook entries with date, description, hours and the mentor's email address.
- Optional evidence upload (PDF, PNG, JPG or WEBP) attached to an entry.
- Submission for verification through a one-time link that expires after seven days.
- A mentor verification page that needs no account, where the mentor approves or rejects the entry.
- Entry statuses (Draft, Pending verification, Verified, Rejected) and resubmission of rejected entries.
- Verified hours shown against each student's required hours.
- A coordinator and lecturer dashboard showing each student's status, verified hours, entries and pending verifications.
- A relational database (SQLite) for users, entries and verification links, with timestamps for each action.

Excluded from the project

- Mentor accounts and a mentor dashboard (mentors verify through the link only).
- Written feedback comments from mentors (the mentor approves or rejects only).
- A separate lecturer dashboard (lecturers use the coordinator dashboard).
- Automatic email or SMS notifications; students send the link themselves.
- Editing or deleting entries after they are saved.
- Reports, data export and analytics.
- Integration with CTU student or learning management systems.
- A native mobile application.
- Use of real student or staff data; all testing uses synthetic data until written consent is obtained (see the Ethical Clearance Confirmation).

1.8 Significance of the Study

The TrackWIL project is expected to provide benefits to several groups involved in the WIL process.

Students

TrackWIL makes it easier for students to manage their daily WIL documentation. Instead of sending documents through email and waiting for them to be returned, students have one place to record their work, request verification and check the status of each entry. They can also see how many verified hours they have completed against the required total.

Workplace Mentors

Mentors can verify a student's work in a few clicks from a link, without creating or remembering an account. Each decision is recorded with a date and time, which provides a clear record of completed mentor approvals.

WIL Coordinators

Coordinators have a central view of student submissions and verification statuses. Instead of manually checking documents or asking students for updates, they can use the dashboard to identify students with outstanding verifications or no recent verified work. This can reduce the administrative workload and make monitoring more efficient.

Lecturers

Lecturers can monitor student progress from the same dashboard. They can identify students who are up to date and students who are falling behind, which allows them to identify problems earlier and provide support where necessary.

Educational Institutions

TrackWIL can help educational institutions improve the organisation and management of WIL documentation. Centralising information reduces dependence on emails, spreadsheets and separate documents and provides a clearer record of student WIL progress.

Software Engineering Field

The project demonstrates how software engineering can be used to solve a real-world problem in education and workplace training. It applies requirements analysis, database design, authentication, role-based access, secure file handling, token-based verification, version control and software testing.

1.9 Chapter Summary

This chapter introduced the TrackWIL project and the challenges of managing Work-Integrated Learning documentation through email, paper and separate files. Students must complete daily documents, have them signed by workplace mentors and wait for them to be returned, which makes it difficult to track submissions and identify outstanding work.

The chapter presented the research aim, five objectives and five research questions, each linked to an objective, that guide the development of TrackWIL. The proposed solution is a web application in which students record daily entries and evidence, mentors verify entries through a one-time link without an account, and coordinators and lecturers monitor progress from a shared dashboard.

The scope defined what is included in the project and what is excluded, and the significance of the study explained how TrackWIL can benefit students, workplace mentors, coordinators, lecturers and educational institutions. Chapter 2 reviews related research, similar systems and technologies.

CHAPTER 2 - LITERATURE REVIEW

2.1 Introduction

This chapter reviews research, existing software and relevant technologies related to Work-Integrated Learning (WIL) documentation and progress monitoring. The review considers how digital systems support students, workplace mentors, coordinators and lecturers, and identifies features that can inform the design of TrackWIL.

The chapter compares three existing systems: InPlace, Sonia Online and PebblePad/Mahara-style ePortfolio tools, against the project's needs. It also considers web applications, relational databases, authentication and role-based access, and secure document upload. The findings are used to define a practical research gap and to justify TrackWIL's technical direction.

2.2 Existing Research

Work-Integrated Learning connects academic learning with practical workplace experience. Technology can support WIL in different ways: it may deliver learning activities, or it may provide administrative support for placements, communication, evidence collection and assessment. Schuster and Glavas (2017) explain that electronic WIL covers a range of technology uses rather than one single model. For TrackWIL, the administrative role is especially relevant because the project focuses on daily activity records, evidence submission, mentor verification and progress monitoring.

Effective placement supervision depends on communication and coordination between students and supervisors. Hains-Wesson et al. (2026), in a systematic literature review of placement supervision, examine student and supervisor perspectives and identify patterns and gaps in the literature. This supports the need to consider both sides of the submission and verification process rather than designing a system only for students or academic staff.

Industry systems show that placement management commonly involves centralised records, user portals, communication, compliance and reporting. InPlace describes tools for central placement visibility, student portals, time sheets, communications, compliance and analytics (InPlace Software, n.d.). Sonia Online describes itself as a placement management system containing placement information and paperwork for students and host organisations (Sonia Online, n.d.). These examples show that digital coordination is an established approach, although their exact configuration and available functions depend on the institution using them.

Portfolio platforms also provide relevant ideas for collecting and reviewing evidence. Mahara allows learners to upload evidence, reflect on experiences, share portfolios with selected people and receive feedback (Mahara Project, n.d.). PebblePad's ATLAS assessment area supports assessors in accessing and reviewing learners' work (PebblePad, 2026). These features are relevant to TrackWIL's evidence and mentor-review workflow, but a portfolio or assessment platform is not necessarily designed around a specific institution's daily WIL forms and status-tracking requirements.

Overall, the literature and industry examples support a centralised digital workflow, clear communication, evidence management and timely verification. However, TrackWIL must still be evaluated against its specific project requirements, including daily submissions, mentor approval or rejection, role-specific views and a visible list of outstanding work. Product websites describe vendor-provided capabilities; they should not be treated as independent proof that a system will improve outcomes in every setting.

2.3 Existing or Similar Systems

Three relevant systems were reviewed: InPlace, Sonia Online and Mahara/PebblePad. The comparison below is based on publicly available product information, not on hands-on testing. Features may vary by institution, subscription or local configuration.

System 1: InPlace. InPlace is designed for higher-education placement management. Its stated features include centralised placement management, student portals, timesheets and schedules, communications, compliance management, integrations and reporting (InPlace Software, n.d.). Its strength is broad administration and visibility across placements. A limitation for this comparison is that its public description emphasises placement allocation, compliance and programme-level management; it does not establish that every TrackWIL-specific daily logbook or mentor-signing workflow is available in a particular deployment. Lesson for TrackWIL: centralise records and provide role-appropriate views, but prioritise the daily documentation workflow required by the project.

System 2: Sonia Online. Sonia Online is presented as a student placement management system for students and host organisations, with placement information and paperwork available through the system (Sonia Online, n.d.). Its strength is that it directly targets placement coordination and provides access for relevant placement participants. Its limitations cannot be fully assessed from the public landing page alone, and access or features may differ between institutions. Lesson for TrackWIL: make student and mentor access straightforward and keep placement paperwork in one organised location.

System 3: Mahara / PebblePad. Mahara supports evidence-based portfolios, reflection, sharing and feedback (Mahara Project, n.d.). PebblePad's ATLAS area supports access to learner submissions and assessment (PebblePad, 2026). Their strengths are evidence collection, sharing and review. A limitation relative to TrackWIL is that portfolio and assessment workflows may not, by themselves, provide the project's daily WIL status controls, outstanding-work monitoring and coordinator view. Lesson for TrackWIL: connect each uploaded evidence item to a dated activity, a verification status and a progress overview.

Table 5: Comparison of similar systems against TrackWIL's needs

Criterion	InPlace	Sonia Online	Mahara / PebblePad	TrackWIL (this project)
Main purpose	Placement management	Placement information and paperwork	Evidence portfolios and assessment	Daily WIL logbook and verification
Daily activity records	Timesheets (stated)	Not clear from public information	Portfolio pages	Dated entries with hours
Evidence upload	Not clear from public information	Paperwork in the system	Yes	Optional PDF or image per entry
Workplace verification	Not clear from public information	Host organisation access (stated)	Assessor review	One-time link, no mentor account
Staff monitoring	Reporting and analytics	Placement information	Assessor reporting	Dashboard: On track, Awaiting, Overdue
Main limitation for this project	Broad and institution-configured	Features vary by institution	Not built around daily status tracking	Prototype scope; no notifications

Comparative summary. InPlace is strongest as a broad placement-management platform; Sonia Online is a directly relevant placement-information and paperwork system; Mahara and PebblePad provide useful portfolio and assessment patterns. TrackWIL draws on these ideas but focuses its scope on daily WIL activity records, supporting evidence, mentor verification and monitoring by coordinators and lecturers. This is a project-specific design rationale, not a claim that the existing products lack all such features.

2.4 Technology Review

Web-based application. A browser-based application is suitable because students, mentors, coordinators and lecturers may access the system from different locations and devices, and a mentor can open a verification link on any device without installing software. A shared application can present role-specific views while keeping records in one place. Its disadvantages include dependence on network availability, the need to support different screen sizes and the need for secure deployment and maintenance.

Framework. Next.js is a React framework in which pages and server-side logic live in one project, and Server Actions let forms call server functions directly without a separate API layer (Vercel, n.d.). This suits a small team building a working prototype in a short time, because one language (TypeScript) is used for the user interface, the business rules and data access. A separate front end and back end (for example, React with an Express API) would give more flexibility for very large systems, but would add a second project to build, test and deploy.

Database. A relational database is appropriate for the structured relationships between users, logbook entries and verification links. Primary and foreign keys and CHECK constraints keep the data consistent and make it simple to count verified hours and outstanding verifications. SQLite was chosen for the prototype because it is a self-contained, serverless SQL database engine that stores the whole database in a single file (SQLite, n.d.), and Node.js includes a built-in SQLite module, so no database server or extra package has to be installed (OpenJS Foundation, n.d.). This makes the application easy to run on every team member's computer and for the live demonstration. A client-server database such as PostgreSQL would be preferred for a multi-user production deployment; because the schema uses standard SQL, it can be migrated later.

Authentication and role-based access. TrackWIL handles student records and workplace evidence, so users should authenticate before accessing protected information. OWASP recommends storing passwords only as salted hashes produced by a slow, purpose-built function and never in plain text (OWASP Foundation, n.d.a). Authorisation should restrict users to the records and actions needed for their roles, and checks should be enforced on the server, not only by hiding interface elements (OWASP Foundation, n.d.b). For workplace mentors, who are external and verify only occasionally, a single-use, time-limited link with a long random token is a simpler alternative to creating accounts, provided the token cannot be guessed and stops working after use or expiry.

Document upload and storage. Evidence upload is a core feature, but uploaded files create security and storage risks. OWASP recommends allowing only necessary file types, limiting file size, generating safe stored file names that users cannot control, and ensuring only authorised users can download uploads (OWASP Foundation, n.d.c). TrackWIL therefore accepts only PDF and image files, stores them under random names outside the public web folder, and serves them only to the owning student and to staff.

Cloud hosting. Cloud hosting can make a web application accessible over the internet without the project team maintaining physical infrastructure. NIST describes cloud computing as on-demand network access to a shared pool of configurable computing resources (Mell and Grance, 2011). For this project the prototype runs locally on synthetic data; cloud hosting is a future option that would require attention to cost, availability, backups, privacy and the location where student information is stored.

Usability and testing. The application should make submission status easy to understand and minimise unnecessary steps for students and mentors. Testing should cover successful and unsuccessful sign-in, role permissions, input and upload validation, submission and verification status changes, link expiry and reuse, and the monitoring dashboard. Claims of improved efficiency should be based on testing or user feedback rather than assumed in advance.

2.5 Research Gap

Existing placement-management systems and portfolio platforms already provide useful functions such as centralised placement information, paperwork, evidence sharing, assessment, communication and reporting. Therefore, the research gap is not that no digital WIL or placement tools exist.

The project-specific gap addressed by TrackWIL is the need for a focused, lightweight workflow aligned with the requirements described in Chapter 1: students record daily WIL activities and upload evidence; workplace mentors, who are outside the institution, verify each entry without having to create an account; and coordinators and lecturers see each student's verification status and outstanding work in one view.

Public product descriptions do not establish whether one existing platform, in the configuration available to the intended users, matches all these requirements in a simple workflow. The project therefore designs and tests a solution against these requirements rather than claiming that existing products cannot support them. The proposed contribution is a working prototype tailored to the stated use case, with a shared record of entries and their verification status.

The evaluation investigates whether students can submit and have entries verified more simply and whether staff can identify outstanding work more easily. This gap and evaluation focus align with the aim and objectives in Chapter 1.

2.6 Conceptual / Technical Direction

TrackWIL follows a centralised, role-based workflow. A student creates a dated logbook entry, records the hours worked, names the workplace mentor and may attach evidence. The student submits the entry, and the system issues a one-time verification link. The mentor opens the link and approves or rejects the entry. The system records the status and timestamps so the student can see the outcome, while coordinators and lecturers use the dashboard to identify students who are on track, awaiting verification or overdue.

The main conceptual entities are User (student, coordinator or lecturer), Logbook Entry and Verification Token. Each entry belongs to one student, and each verification token belongs to one entry, so that every approval or rejection can be traced to the link that produced it. Storing the status on the entry itself avoids multiple conflicting copies of the same submission.

Technical priorities are usability, data integrity, access control, safe file handling, maintainability and testability. Based on the technology review, the team selected Next.js with TypeScript for the application, Tailwind CSS for styling and SQLite for the database. Regardless of the stack, the system validates input and enforces permissions on the server, restricts file uploads and is tested for each role's workflow.

2.7 Chapter Summary

This chapter reviewed research on electronic WIL and placement supervision, and compared InPlace, Sonia Online and portfolio and assessment tools such as Mahara and PebblePad. The review found useful patterns in centralised placement information, evidence collection, submission review, compliance and reporting. It also reviewed the technologies used to build TrackWIL: a Next.js web application, an SQLite relational database, salted password hashing with role-based access, link-based mentor verification and restricted file uploads.

The research gap is a focused workflow that matches TrackWIL's needs for daily activity logging, evidence submission, account-free mentor verification and progress monitoring. These findings inform the requirements in Chapter 3 and the design in Chapter 4. TrackWIL's effectiveness must be established through testing and evaluation rather than assumed.

CHAPTER 3 - REQUIREMENTS & METHODOLOGY

3.1 Introduction

This chapter explains what TrackWIL must do, how those requirements were identified and how the system was developed. TrackWIL is a web-based system that replaces the paper, PDF and email process for Work-Integrated Learning (WIL) logbooks.

In the current process students complete and sign daily WIL documents, send them to workplace mentors for verification and wait for the signed documents to be returned. This manual process can lead to delays, lost documents and difficulty tracking student progress. TrackWIL provides a central platform where students record daily work, workplace mentors verify entries through a secure link, and coordinators and lecturers monitor student progress.

The chapter describes the stakeholders, the functional and non-functional requirements, the requirements-gathering process, the development methodology, the use cases and user stories, the system architecture, and the ethical, privacy and security considerations. The complete requirements are specified in the Software Requirements Specification (SRS), and Chapter 5 traces every requirement to its test cases.

3.2 Stakeholders and Users

Stakeholders are people or organisations affected by TrackWIL, while users are the people who interact directly with the system.

Table 6: Stakeholders, their needs and how they use TrackWIL

Stakeholder	Main need	How they use TrackWIL	Access
Student (primary user)	Record daily work and get it verified without chasing paper or email	Signs up, creates logbook entries, uploads evidence, submits entries for verification, tracks verified hours	Account (student role)
Workplace mentor (primary user)	Confirm a student's work quickly, without another account to manage	Opens the link sent by the student and approves or rejects the entry	No account; one-time link valid for 7 days
WIL coordinator (primary user)	Know which students are behind or waiting on a mentor	Monitors every student's status, verified hours and pending verifications	Account (coordinator role)
Lecturer (primary user)	Identify students falling behind so that support can be given	Uses the same monitoring dashboard as the coordinator	Account (lecturer role)
Training institution (CTU)	Reliable WIL records and protection of personal information	Benefits from central records; sets data and consent rules	None
Development team	Clear, testable requirements	Builds, tests and maintains the system	Repository and development data
Project supervisor (client)	A project that meets its objectives and the assessment rubric	Reviews weekly submissions and gives feedback	Repository and demonstration

3.3 Functional Requirements

Each functional requirement has an ID, a description and a priority. **High** means the requirement is part of the core product that must work for the demonstration; **Medium** means it adds important value but the core workflow can be shown without it. Each requirement is testable and traced to test cases in Chapter 5.

Table 7: Functional requirements

ID	Requirement	Priority
FR-01	The system must allow a registered user to sign in with an email address and password and must send them to the dashboard for their role.	High
FR-02	The system must allow a student to create an account with a CTU student email address (eight-digit student number @ctucareer.co.za) and a password of at least 8 characters with an uppercase letter, a lowercase letter, a number and a special character.	Medium
FR-03	The system must restrict each page and action to the roles allowed to use it: the student area to students, the monitoring dashboard to coordinators and lecturers, and the verification page to holders of a valid link.	High
FR-04	A student must be able to create a daily logbook entry with a work date, a description, the hours worked (more than 0 and at most 24) and the workplace mentor's email address.	High
FR-05	A student must be able to attach one optional evidence file (PDF, PNG, JPG or WEBP) to an entry.	Medium
FR-06	A student must be able to submit a Draft or Rejected entry for verification; the system must create a unique link that expires after 7 days and set the entry to Pending verification.	High
FR-07	A workplace mentor must be able to open the link without an account, see the student's name, date, hours and description, and approve or reject the entry.	High
FR-08	A verification link must work only once and only before it expires; an invalid, used or expired link must show a message and must not change the entry.	High
FR-09	The system must show each entry's status as Draft, Pending verification, Verified or Rejected.	High
FR-10	A student must be able to view all of their entries, newest first, and their verified hours against their required hours.	High
FR-11	Coordinators and lecturers must be able to see every student with their status (On track, Awaiting verification or Overdue), verified hours, number of entries and number of pending verifications. A student is Overdue when they have no verified entry in the last 14 days.	High
FR-12	The system must automatically record when each entry is created, updated and verified, and when each link is used and with what decision.	Medium
FR-13	Evidence files must be downloadable only by the student who uploaded them, coordinators and lecturers.	High
FR-14	A signed-in user must be able to sign out, which ends their session.	High

During development the team reviewed the original requirements against the time available. The following requirements from the Week 3 draft were deferred and are recorded as future work, so that the core workflow could be completed and tested to a good standard.

Table 8: Requirements deferred to future versions

Deferred requirement	Reason	Current behaviour
Mentor accounts and a mentor dashboard	Mentors are external, verify occasionally and should not need another account	Mentors verify through the one-time link
Written mentor feedback on rejected entries	Time; the core decision (approve or reject) was prioritised	The mentor approves or rejects; the student sees the Rejected status
Mentor access to evidence files	Evidence is protected by sign-in; a link-based download needs its own security design	Evidence is visible to the student and to staff
Separate lecturer dashboard	Coordinators and lecturers need the same information	Lecturers use the coordinator dashboard
Automatic email notifications	Requires an email service and consent for contact details	The student copies the link and sends it to the mentor
Reports and data export	Stretch goal in the proposal	Not implemented

3.4 Non-Functional Requirements

Each non-functional requirement states how it will be measured, so that it can be tested rather than judged by opinion.

Table 9: Non-functional requirements

ID	Category	Requirement	How it is measured
NFR-01	Usability	A first-time student can create and submit an entry for verification in under 3 minutes without help.	Timed task with test users (Chapter 5)
NFR-02	Performance	Each page loads and each action completes within 2 seconds on the test computer with the demonstration data.	Browser timing during system tests
NFR-03	Security	Passwords are stored only as salted scrypt hashes; the session cookie is HttpOnly; verification tokens contain at least 128 bits of randomness; all role checks run on the server.	Code review and security test cases
NFR-04	Reliability and data integrity	Database constraints reject invalid data (unknown status, hours outside 0–24, entries without a student), and multi-step updates run in a transaction so that no partial update is saved.	Database constraint tests
NFR-05	Maintainability	The code is written in TypeScript in separate layers (pages, actions, repository, database), passes the linter, and the README explains how to run it in no more than five commands.	Lint run and README walkthrough

ID	Category	Requirement	How it is measured
NFR-06	Compatibility	The application works in current versions of Google Chrome and Microsoft Edge and is usable on a 375-pixel-wide mobile screen and a 1280-pixel desktop screen.	Cross-browser and responsive tests
NFR-07	Privacy	Only the data needed for WIL tracking is stored (name, email, entries, evidence); testing and demonstrations use synthetic data only.	Data review against the Ethical Clearance Confirmation

3.5 Requirements Gathering

The requirements were gathered and refined through the methods below. Interviews, questionnaires and usability tests with staff and students are planned, but, as stated in the Ethical Clearance Confirmation, no personal information may be collected before written campus permission is granted. Until then, the team relied on document analysis, comparison with existing systems, supervisor feedback and review of the working prototype, and all testing uses synthetic data.

Table 10: Requirements-gathering methods and evidence

Method	What was done	Evidence	Effect on requirements
Scenario and document analysis	Analysed the current paper, PDF and email logbook process and the module brief	Project proposal; Chapter 1	Defined the problem, the four user groups and the core workflow
Review of similar systems	Compared InPlace, Sonia Online and Mahara/PebblePad	Chapter 2; Literature Review Matrix	Mentor verification without an account; status-based monitoring
Supervisor (client) feedback	The proposal was reviewed by the supervisor and revised on 7 October 2026	Topic Acceptance form; revised proposal in Git	Narrowed the scope to three core features and moved analytics, exports and audit logs to stretch goals
Team workshops	Weekly group meetings to agree and prioritise requirements	Attendance register; Weekly Progress Report	High and Medium priorities; deferred requirements
Prototype review	The Sprint 1 prototype was demonstrated to the team and checked against the requirements	Git history (12–15 October 2026)	Added sign-up rules, the 24-hour limit and lecturer access
Planned: questionnaire and usability test	Short task-based test and questionnaire with volunteer students and staff once permission is granted	Ethical Clearance Confirmation	Will evaluate NFR-01 and overall usefulness (Chapter 5)

The requirements were then reviewed to make sure that each one is clear, testable and linked to a research objective, and were recorded in the SRS.

3.6 Development Methodology

TrackWIL was developed with a hybrid Agile approach based on Scrum, using one-week sprints that end on each Friday submission date (Schwaber and Sutherland, 2020). Each sprint followed the same cycle: plan the sprint backlog, design, build, test, and review the result with the team and the supervisor's feedback.

Table 11: Sprint plan

Sprint	Dates	Goal	Main deliverables
Sprint 0	2–8 October 2026	Proposal, scope and requirements	Approved topic; core and stretch scope; ethics form
Sprint 1	9–15 October 2026	Logbook core	Chapters 1–2; Next.js prototype with sign-in, logbook entries, evidence upload and hour progress
Sprint 2	16–22 October 2026	Verification and monitoring	Chapters 3–4; SRS; token verification; coordinator dashboard; sign-up; SQLite database
Sprint 3	23–29 October 2026	Test, evaluate and submit	Chapters 5–6; testing workbooks; defect fixes; presentation and live demonstration

Justification for Agile

Agile suits TrackWIL for four reasons. First, the system has three features that can be built and tested separately (logbook, verification and dashboard), so each sprint can deliver something that works. Second, the requirements changed during the project: supervisor feedback narrowed the scope, and the prototype review added sign-up rules and lecturer access. An iterative approach accepts such changes, as described in the Agile Manifesto's preference for "responding to change over following a plan" (Beck et al., 2001). Third, the weekly submissions act as natural sprint reviews with the supervisor. Fourth, the four team members can work on different features in parallel on separate Git branches.

Alternative considered: Waterfall

The Waterfall model completes each phase (requirements, design, implementation, testing) before the next starts (Sommerville, 2016). It works well when requirements are stable and well understood. It was rejected because TrackWIL's requirements were still changing, the project had only four weeks, and Waterfall would have left all testing until the end, when there would be no time to fix defects. With Agile, each feature was tested in the sprint in which it was built.

3.7 Use Cases / User Stories

The use cases are shown in the use case diagram in Chapter 4. Each user story has acceptance criteria that are used as the basis for the test cases in Chapter 5.

Table 12: User stories and acceptance criteria

ID	User story	Acceptance criteria	FRs
US-01	As a student, I want to sign up and sign in securely so that I can access my own logbook.	Valid CTU email and strong password create an account; wrong password shows "Invalid email or password."	FR-01, FR-02, FR-14

ID	User story	Acceptance criteria	FRs
US-02	As a student, I want to record a day's work so that my workplace activities are documented in one place.	Date, description and hours (0–24) are required; the saved entry appears as Draft.	FR-04, FR-09, FR-12
US-03	As a student, I want to attach evidence so that my work can be checked.	PDF or image accepted; other file types rejected; only the student and staff can open the file.	FR-05, FR-13
US-04	As a student, I want to send my entry to my mentor for verification so that my hours count.	A link valid for 7 days is shown; the entry becomes Pending verification.	FR-06
US-05	As a workplace mentor, I want to approve or reject an entry from a link so that I do not need an account.	The link shows the entry; Approve sets Verified and Reject sets Rejected; the link cannot be used again.	FR-07, FR-08
US-06	As a student, I want to see the status of my entries and my verified hours so that I know what is outstanding.	Statuses are shown on the logbook; verified hours are shown against the required hours; a rejected entry can be resubmitted.	FR-09, FR-10
US-07	As a coordinator, I want to see which students are on track, awaiting verification or overdue so that I can follow up.	The dashboard lists every student with status, verified hours, entries and pending count.	FR-11
US-08	As a lecturer, I want to view student progress so that I can support students who are falling behind.	A lecturer signs in to the same dashboard and cannot open the student area.	FR-03, FR-11

Main use case: verify a logbook entry (UC-05)

- **Actors:** Student (initiates), workplace mentor (verifies).
- **Precondition:** The student has a Draft or Rejected entry with a mentor email address.
- **Main flow:** (1) The student submits the entry for verification. (2) The system creates a one-time link valid for 7 days and sets the entry to Pending verification. (3) The student sends the link to the mentor. (4) The mentor opens the link and reviews the entry. (5) The mentor selects Approve or Reject. (6) The system marks the link as used and sets the entry to Verified or Rejected.
- **Alternative flow:** If the link is invalid, already used or expired, the system shows a message and does not change the entry.
- **Postcondition:** The entry's status and verification time are stored, and the coordinator dashboard reflects the change.

3.8 System Architecture

TrackWIL uses a three-layer architecture inside a single Next.js application. The detailed architecture diagram is presented in Chapter 4.

Presentation Layer

The presentation layer is the part of the system that users interact with through a web browser. It provides the sign-in and sign-up pages, the student logbook, the mentor verification page and the coordinator and lecturer dashboard. It is built with React components and styled with Tailwind CSS.

Application Layer

The application layer handles the business rules through Next.js Server Actions: authentication, role checks, input validation, creating entries, issuing verification links and recording mentor decisions. This layer makes sure that users can perform only the actions allowed for their role; for example, a student cannot open the coordinator dashboard and cannot verify their own entry, because verification requires the secret link.

Data Layer

The data layer stores user accounts, logbook entries and verification links in an SQLite relational database, and evidence files in a protected folder on the server. A repository module is the only part of the code that runs SQL, which keeps the database separate from the rest of the application.

Communication Between the Layers

When a student submits an entry, the presentation layer calls a Server Action in the application layer. The action checks that the user is signed in and owns the entry, validates the request and saves the change through the data layer, then returns the result so that the page can show the new status. The mentor's approval follows the same path, starting from the verification link.

3.9 Ethical, Privacy and Security Considerations

User Authentication and Access Control

TrackWIL requires students, coordinators and lecturers to sign in. Passwords are stored only as salted hashes (OWASP Foundation, n.d.a). Role checks run on the server for every protected page and action, so that hiding a button is never the only protection (OWASP Foundation, n.d.b). Mentors use a long random link that works only once and only for seven days.

Data Accuracy and Accountability

The system records when each entry is created and verified and which link was used for each decision, so the history of an entry is clear. Verification is always performed by the mentor through the link; entries are never approved automatically, and a student cannot change an entry's status themselves.

Consent and Responsible Data Use

No personal information about real staff or students is collected until written campus permission is granted. Participation in any later questionnaire or usability test will be voluntary, and participants will be told the purpose of the data collection and who can see it. Until then, all development, testing and demonstrations use synthetic data.

Privacy and Protection of Personal Information

The system collects only the information necessary for WIL tracking. Personal information must be handled in line with institutional policy and the Protection of Personal Information Act (POPIA) (Republic of South Africa, 2013). Evidence files are stored under random names and can be opened only by the student who uploaded them and by staff.

Security Risks Identified

The team identified security risks during design and testing, including the risk of uploading harmful files (reduced by allowing only PDF and image types and storing files under random names) and the risk of a session cookie being reused or altered. These risks and their treatment are recorded in the Risk Register and evaluated in Chapter 5.

3.10 Chapter Summary

This chapter identified TrackWIL's stakeholders and specified fourteen prioritised functional requirements and seven measurable non-functional requirements, together with the requirements that were deferred to future versions. It explained how the requirements were gathered within the limits of the ethical clearance, justified the use of a hybrid Agile (Scrum) approach over Waterfall, and presented eight user stories with acceptance criteria and the main verification use case. Finally, it described the three-layer architecture and the ethical, privacy and security considerations. Chapter 4 presents the detailed design and implementation.

CHAPTER 4 - SYSTEM DESIGN & DEVELOPMENT

4.1 Introduction

This chapter shows how the requirements in Chapter 3 were turned into a design and a working application. It presents the system architecture, the UML diagrams (use case, class, activity and sequence), the database design, the user interface, the development environment, the implementation of the key modules, and the version-control evidence from the team's GitHub repository.

TrackWIL replaces the PDF and email logbook with a web application. Students log workplace hours and evidence, the workplace mentor verifies each entry through a one-time link without needing an account, and the WIL coordinator and lecturers can see who is on track, who is waiting on a mentor and who is overdue. The source code is in the folder 05_Source_Code/trackwil of the repository.

4.2 System Architecture

TrackWIL is a single Next.js application that runs in one Node.js process. The browser communicates with this process over HTTP (HTTPS when hosted). The process serves the React pages, runs the business rules in Server Actions, and reads and writes the SQLite database. Uploaded evidence files are stored on the server's disk. Figure 1 shows the architecture.

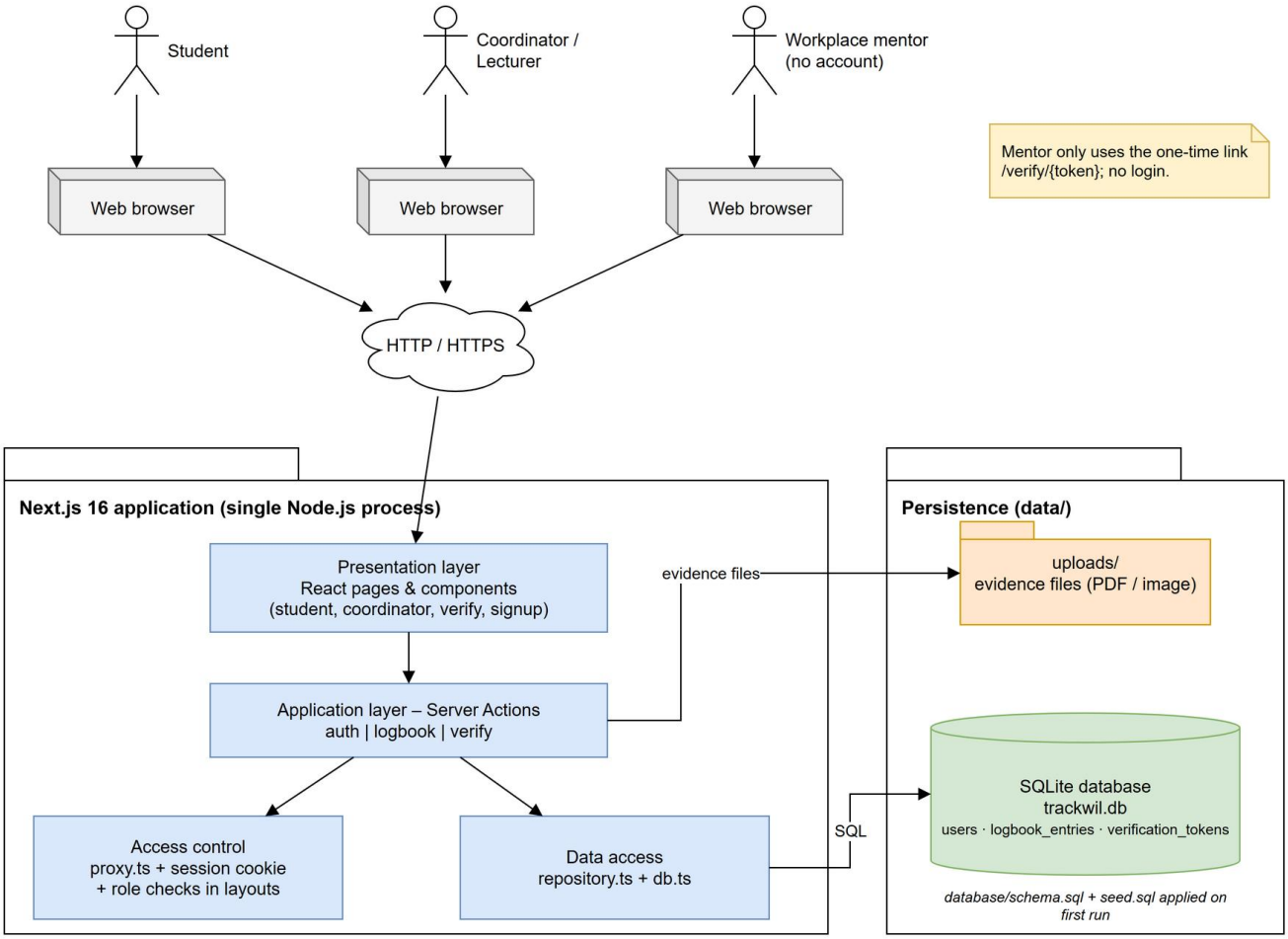


Figure 1: TrackWIL system architecture

The architecture has three logical layers. The presentation layer contains the pages and components each user sees. The application layer contains the Server Actions that validate input, check the user's role and apply the business rules. The data layer contains the repository and database modules; the repository is the only code that runs SQL, so the rest of the application never depends on how the data is stored. This separation is what allowed the team to replace the first prototype's JSON files with SQLite by changing only the data layer.

Table 13: Main components and their responsibilities

Component	Location in code	Responsibility	Requirements
Sign-in and sign-up pages	src/app/page.tsx, src/app/signup	Collect credentials; show validation errors	FR-01, FR-02
Student area	src/app/student	Logbook, new entry form, entry details, verification link	FR-04 to FR-06, FR-09, FR-10
Mentor verification page	src/app/verify/[token]	Show the entry and Approve / Reject buttons without sign-in	FR-07, FR-08
Coordinator dashboard	src/app/coordinator	Table of all students with status and hours (also used by lecturers)	FR-11
Server Actions	src/app/actions (auth, logbook, verify)	Validation, role checks and business rules	FR-01 to FR-08, FR-14
Access control	src/proxy.ts, layouts, src/lib/session.ts	Redirect signed-out users; enforce roles	FR-03
Evidence route	src/app/api/evidence/[fileName]	Serve evidence only to the owner and staff	FR-13
Repository and database	src/lib/repository.ts, src/lib/db.ts	All SQL queries and transactions; schema and seed on first run	FR-12, NFR-04
Metrics	src/lib/logbook-metrics.ts	Verified hours and On track / Awaiting / Overdue status	FR-10, FR-11

Data flow example (submitting an entry). The student clicks Submit for verification. The browser calls the submitEntryForVerificationAction Server Action, which loads the signed-in user, checks that the student owns the entry and that it is a Draft or Rejected entry with a mentor email, creates a random token and asks the repository to save the token and change the entry's status in one transaction. The page then shows the new status and the link.

Technology stack justification. The table below summarises the technologies chosen and why. The comparison of alternatives is discussed in Chapter 2, Section 2.4.

Table 14: Technology stack and justification

Layer	Technology	Why it was chosen
Front end	React 19 with Tailwind CSS 4	Component-based pages; utility classes give a consistent, responsive layout quickly
Application	Next.js 16 (App Router, Server Actions) with TypeScript 5	One project and one language for pages and server logic; type checking catches errors early
Database	SQLite through Node.js's built-in node:sqlite module	Real relational database with keys and constraints, but no server to install; one file that is easy to reset for demonstrations
Security	Node.js crypto (scrypt, randomBytes)	Standard, well-tested functions for password hashing and unguessable tokens; no extra dependencies
Version control	Git and GitHub	Branches, pull requests and a full history of each member's work

4.3 UML Design

Four UML diagrams describe the system from different views: the use case diagram (what each user can do), the class diagram (the main data types and code modules), the activity diagram (the end-to-end workflow) and the sequence diagram (the messages exchanged during mentor verification). The diagrams were drawn in draw.io, and their source files are in 04_System_Design_and_Development/diagrams.

Use Case Diagram

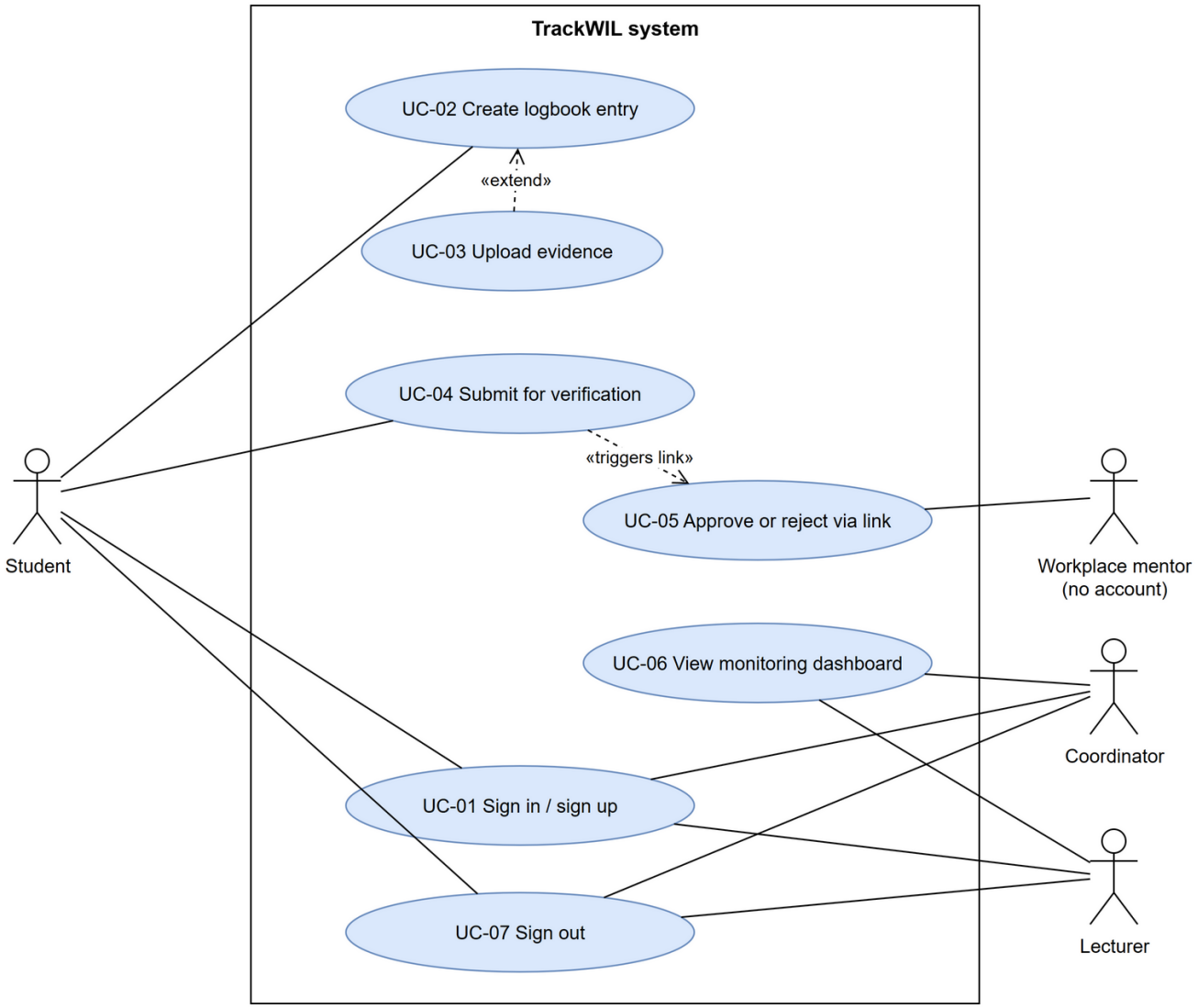


Figure 2: Use case diagram

Figure 2 shows four actors. The student signs in or signs up (UC-01), creates logbook entries (UC-02), optionally uploads evidence (UC-03, which extends UC-02 because it is optional), submits entries for verification (UC-04) and signs out (UC-07). Submitting an entry triggers the link that the workplace mentor uses to approve or reject the entry (UC-05); the mentor has no account and takes part in no other use case. The coordinator and the lecturer both sign in, view the monitoring dashboard (UC-06) and sign out. Each use case maps to the functional requirements listed in the SRS, Section 5.

Class Diagram

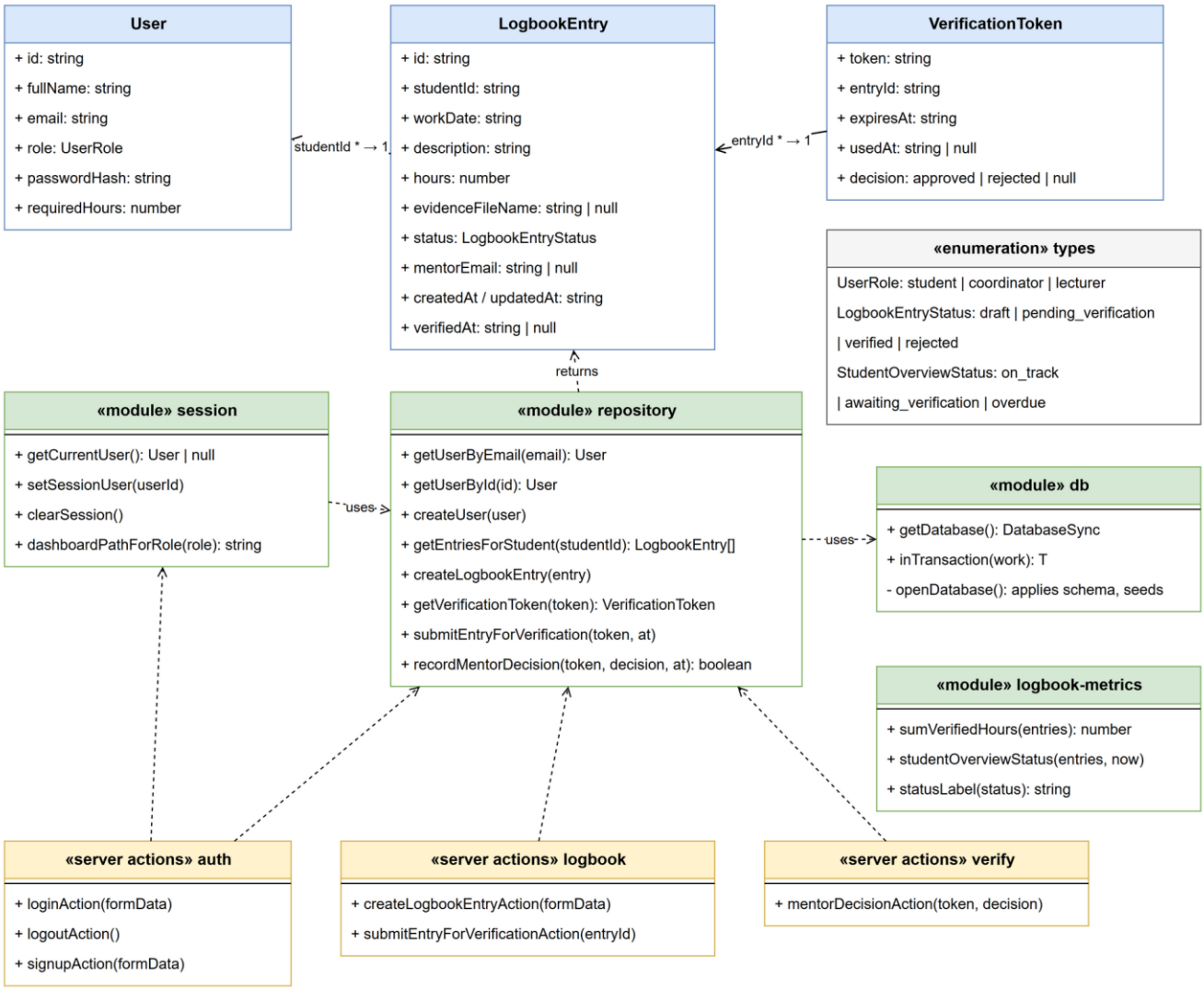


Figure 3: Class diagram of the main types and modules

Figure 3 shows the three domain types that are stored in the database (User, LogbookEntry and VerificationToken) and the enumerations they use. A LogbookEntry belongs to one User through studentId, and a VerificationToken belongs to one LogbookEntry through entryId. The lower half shows how the code is organised: the Server Actions (auth, logbook and verify) depend on the session and repository modules, the repository depends on the db module, and the logbook-metrics module calculates verified hours and the dashboard status from a student's entries. Because Next.js code is organised in modules and functions rather than classes, modules are shown with the «module» and «server actions» stereotypes.

Activity Diagram

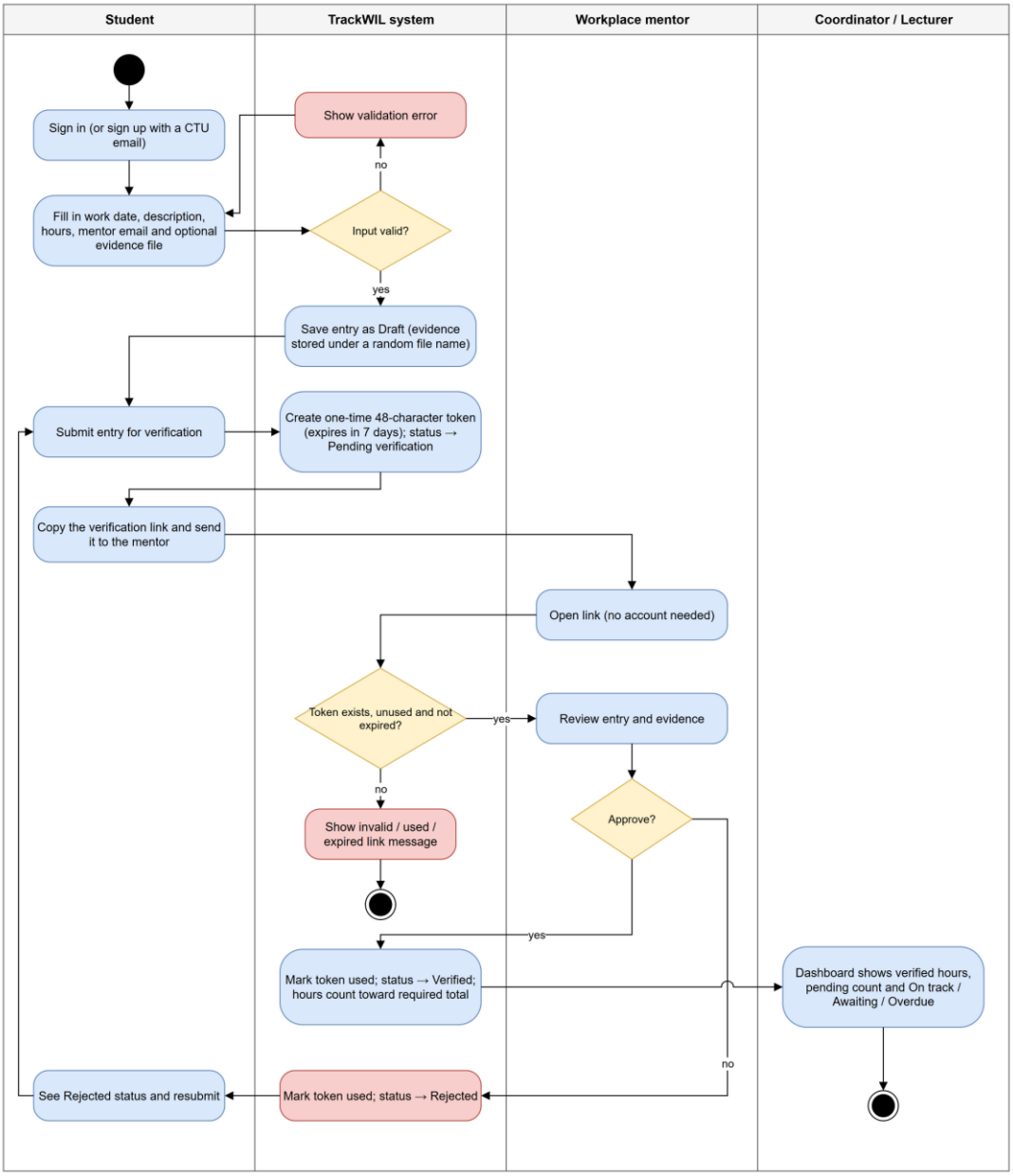


Figure 4: Activity diagram of the logbook and verification workflow

Figure 4 follows one logbook entry from creation to verification across four swimlanes. Invalid input returns the student to the form. A valid entry is saved as a Draft, then submitted, which creates a token and sets the status to Pending verification. When the mentor opens the link the system checks that the token exists, is unused and has not expired; if not, an error is shown and nothing changes. An approval sets the entry to Verified and adds its hours to the student's total on the dashboard. A rejection sets it to Rejected, and the student can resubmit it, which creates a new token.

Sequence Diagram

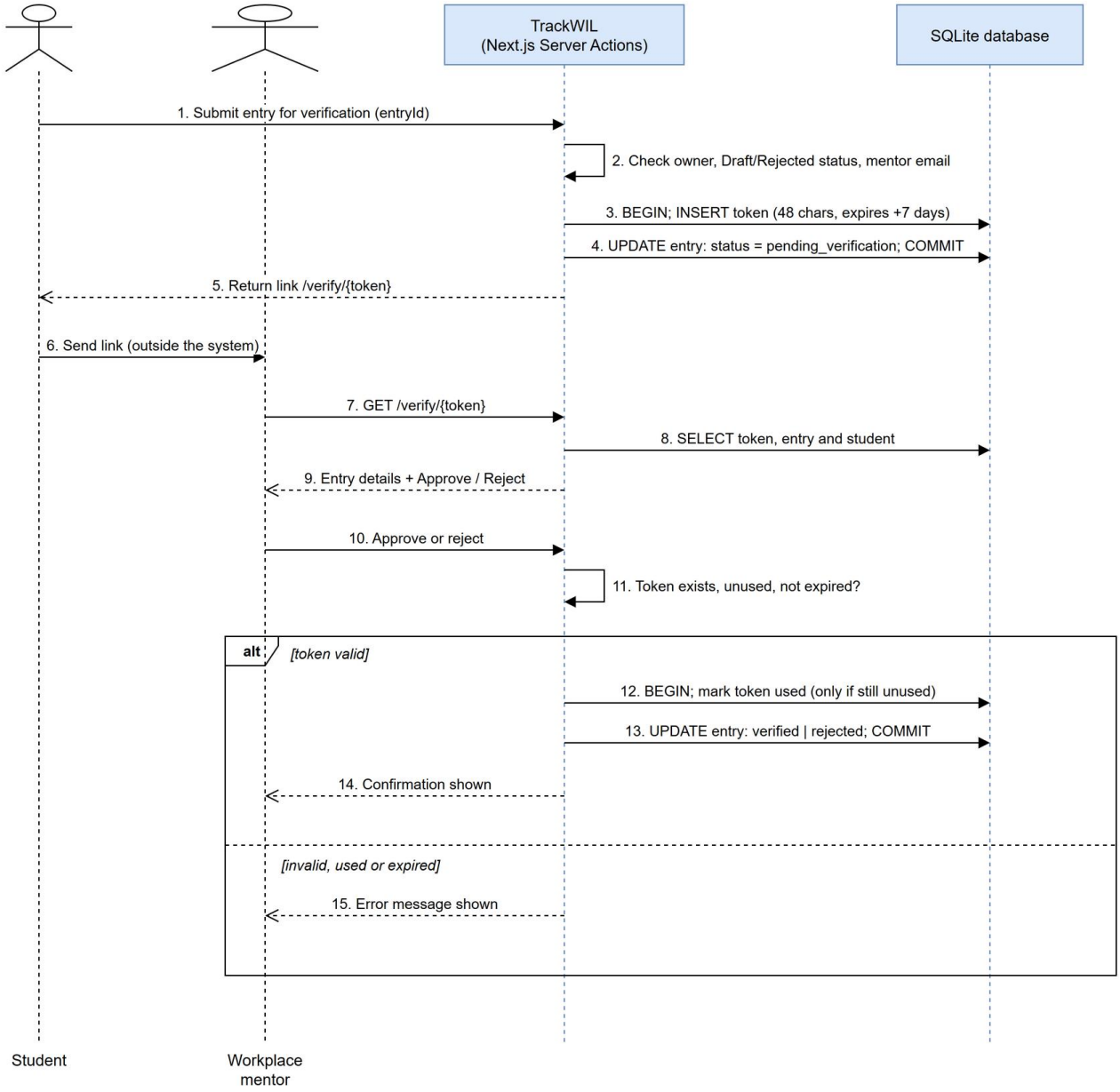


Figure 5: Sequence diagram of mentor verification

Figure 5 shows the messages between the student, the mentor, the TrackWIL server and the SQLite database during verification. Steps 3–4 and 12–13 each run inside a database transaction, so a token is never saved without the matching status change. Step 12 updates the token only if it is still unused, which prevents two decisions being recorded if the mentor clicks twice or opens the link in two tabs. The alt frame shows the two outcomes of the token check.

4.4 Database Design

TrackWIL uses an SQLite relational database with three tables. The schema is in 05_Source_Code/trackwil/database/schema.sql and the synthetic seed data is in seed.sql; both are applied automatically the first time the application starts. Figure 6 shows the entity-relationship diagram, and the full data dictionary is in the Database Design document.

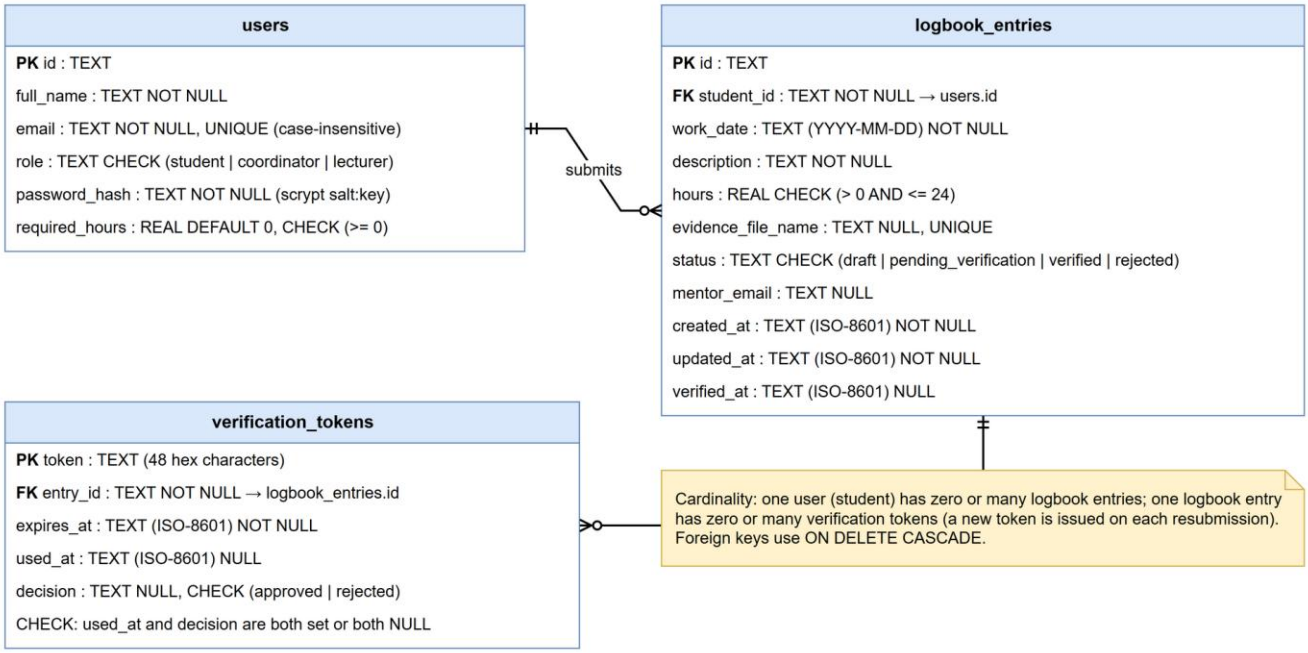


Figure 6: Entity-relationship diagram (SQLite)

Entities and keys. users stores every account (students, coordinators and lecturers), with id as the primary key and a unique, case-insensitive email. logbook_entries stores one row per day of work, with id as the primary key and student_id as a foreign key to users. verification_tokens stores one row per verification link, with the token itself as the primary key and entry_id as a foreign key to logbook_entries.

Relationships and cardinality. One student has zero or many logbook entries, and each entry belongs to exactly one student. One entry has zero or many verification tokens (a new token is created each time a rejected entry is resubmitted), and each token belongs to exactly one entry. Both foreign keys use ON DELETE CASCADE, so deleting a user cannot leave orphaned entries or tokens.

Normalisation. The design is in third normal form. Every column holds a single value (1NF); every table has a single-column primary key, so no column depends on only part of a key (2NF); and no non-key column depends on another non-key column (3NF). For example, the student's name is stored once in users and not copied into each entry. Values that can be calculated, such as a student's verified hours and their On track / Awaiting / Overdue status, are not stored at all but calculated from the entries when the dashboard is displayed, so they can never be out of date. The mentor's email is stored on the entry because the mentor belongs to the placement for that piece of work and mentors have no account of their own.

Integrity rules. CHECK constraints limit role and status to their allowed values, require hours to be more than 0 and at most 24, and require a token's used_at and decision to be set together. The application validates the same rules first and shows a friendly message; the database constraints are a second line of defence.

Design trace. The database design supports FR-04 (entry fields), FR-06 to FR-08 (tokens with expiry, use time and decision), FR-11 (status calculated from entries) and FR-12 (timestamps).

4.5 User Interface Design

The user interface was designed as a working prototype rather than paper wireframes: each screen was built in the first sprint, reviewed by the team and refined in the next sprint. The screenshots below were captured from the application running with the synthetic demonstration data.

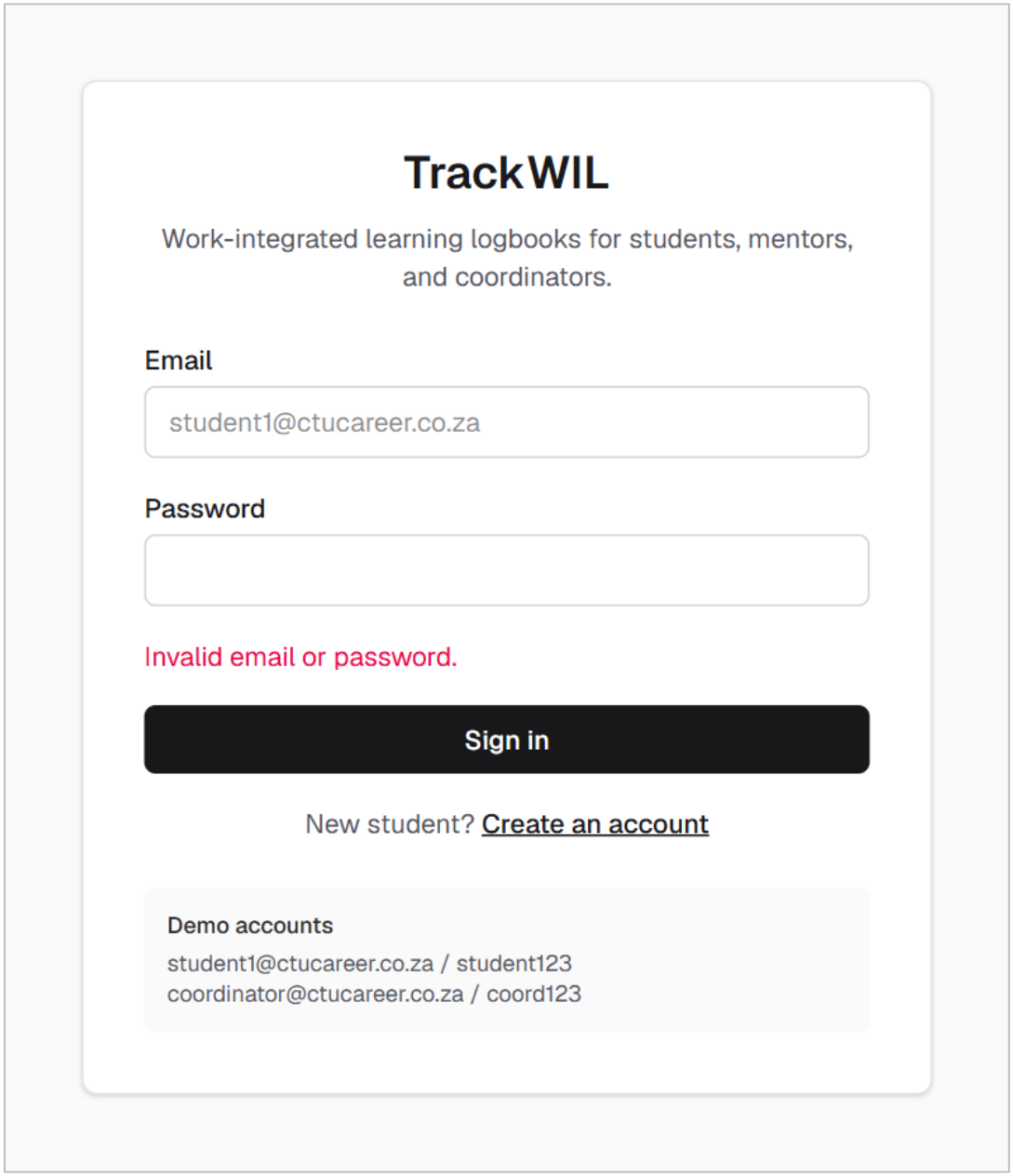


Figure 7: Sign-in page with an error for a wrong password

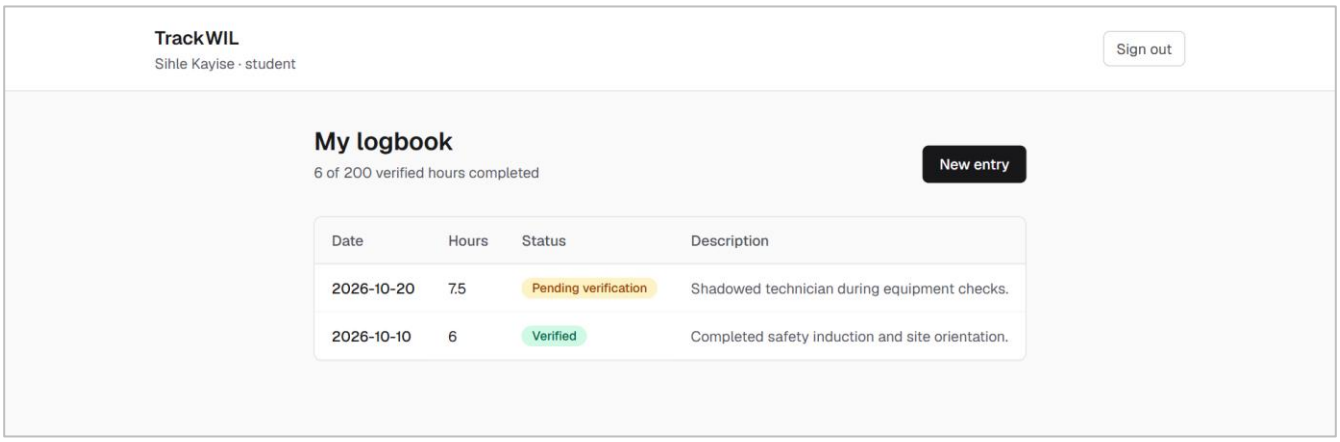


Figure 8: Student logbook showing verified hours and entry statuses

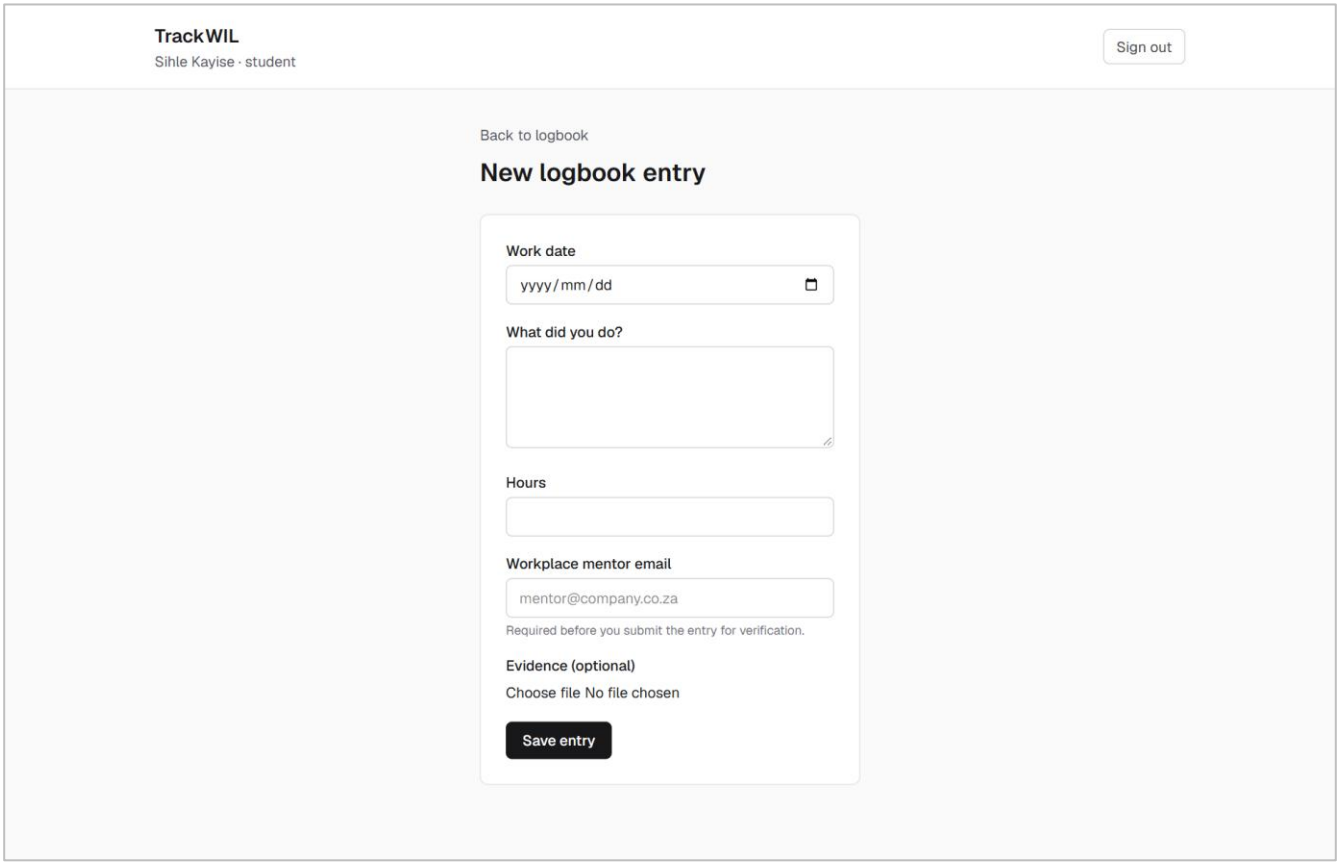


Figure 9: New logbook entry form

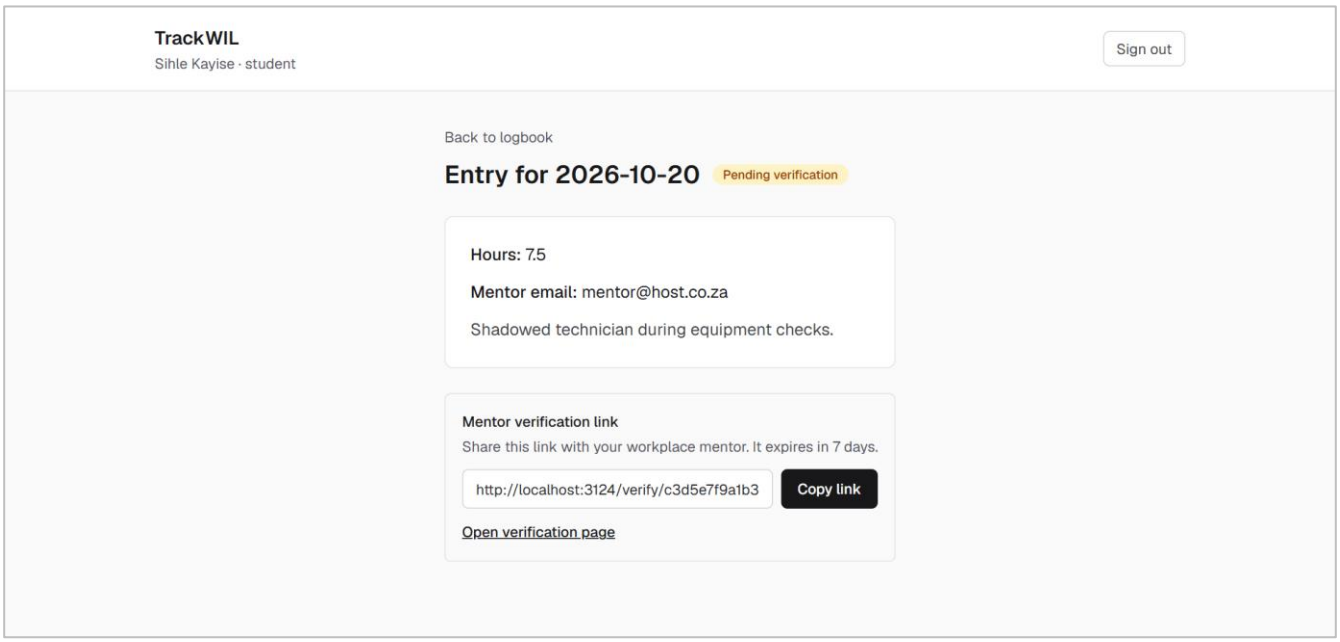


Figure 10: Entry details with the mentor verification link

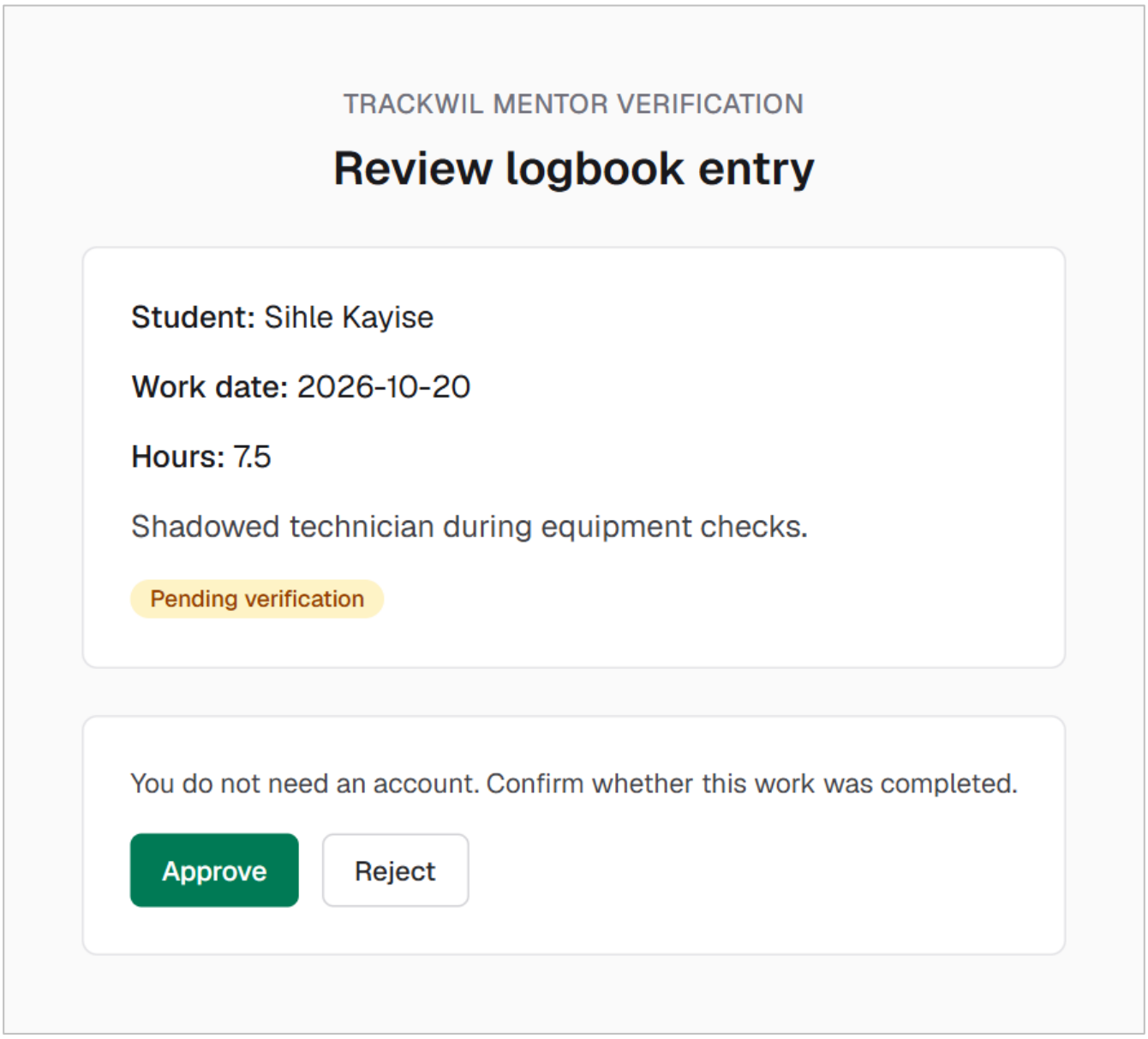


Figure 11: Mentor verification page (no sign-in required)

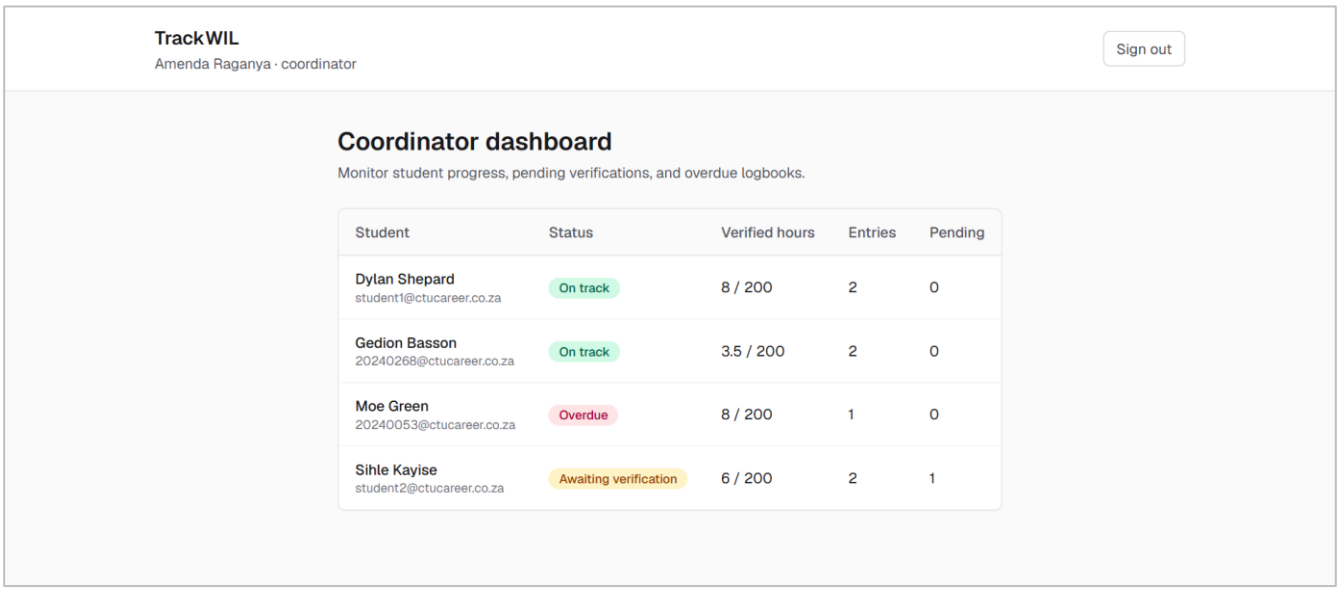


Figure 12: Coordinator dashboard with On track, Awaiting verification and Overdue statuses

Table 15: Usability decisions

Decision	Reason	Screen
Plain-language error messages next to the form (for example "Invalid email or password.")	Users can see what went wrong and correct it without leaving the page	Sign-in, sign-up, new entry
Each status shown as a coloured badge with a text label	Colour makes status quick to scan; the text means it does not rely on colour alone	Logbook, entry details, dashboard
Progress shown as "X of 200 verified hours completed" at the top of the logbook	The student's most important question is answered first	Logbook
Only four fields on the new-entry form, with evidence marked as optional and a hint under the mentor email	Keeps daily logging quick (NFR-01)	New entry
Copy link button and a note that the link expires in 7 days	The student can paste the link into an email or chat without retyping it	Entry details
Mentor page needs no sign-in and has two clear buttons	Mentors are busy and external; verification should take seconds	Mentor verification
One table on the dashboard with status first, then hours and pending count	Coordinators can see at a glance which students need follow-up	Coordinator dashboard
The header always shows the user's name, role and a Sign out button	Users always know which account they are using	All signed-in screens

4.6 Development Environment

Table 16: Development environment

Item	Tool or version
Languages	TypeScript; HTML and CSS through JSX and Tailwind; SQL (schema and queries)
Framework	Next.js 16.4 (App Router, Server Actions) with React 19.3

Item	Tool or version
Runtime	Node.js 24 (minimum 22.13 for the built-in node:sqlite module)
Database	SQLite (file data/trackwil.db), schema in database/schema.sql
Libraries	Tailwind CSS 4; Node.js crypto, fs and node:sqlite (built in)
Code quality	TypeScript compiler (type checking) and ESLint 9 with the Next.js rules
IDE	Visual Studio Code
Design tools	draw.io (UML, ERD and architecture diagrams)
Version control	Git and GitHub (https://github.com/Noobey32/TrackWIL)
Cloud services	None; the prototype runs locally
Operating system	Microsoft Windows 11

To run the application, install Node.js 22.13 or newer and run npm install and npm run dev in 05_Source_Code/trackwil, then open http://localhost:3000. The README lists the demonstration accounts. The command npm run db:reset restores the demonstration data.

4.7 Implementation

This section explains the most important modules. Short code excerpts are included only where they show how a requirement or security rule is enforced.

Project Structure

Table 17: Source code structure (05_Source_Code/trackwil)

Folder or file	Contents
database/	schema.sql (tables, keys, constraints) and seed.sql (synthetic data)
scripts/reset-db.mjs	Rebuilds the database from the SQL scripts
src/app/	Pages and routes: sign-in, signup, student, coordinator, verify, api/evidence
src/app/actions/	Server Actions: auth.ts, logbook.ts, verify.ts
src/components/	Forms, status badges, header, copy-link button
src/lib/	db.ts, repository.ts, session.ts, password.ts, signup-validation.ts, logbook-metrics.ts, types.ts
src/proxy.ts	Redirects signed-out visitors away from protected areas

Authentication and Passwords

Passwords are never stored. On sign-up the password is hashed with scrypt and a random 16-byte salt, and on sign-in the hash is recalculated and compared in constant time, as recommended by OWASP (OWASP Foundation, n.d.a). Sign-up also checks that the email is a CTU student address and that the password meets the strength rules (FR-02).

```
export function hashPassword(password: string): string {
  const salt = randomBytes(16).toString("hex");
  const derivedKey = scryptSync(password, salt, KEY_LENGTH);
  return `${salt}:${derivedKey.toString("hex")}`;
}
```

After a successful sign-in the server sets an HttpOnly session cookie and redirects the user to /student or /coordinator, depending on their role.

Role-Based Access

Access is checked on the server at two levels. The proxy redirects any visitor without a session away from /student and /coordinator. Each area's layout then loads the user and checks the role, so that a coordinator cannot open the student area and a student cannot open the dashboard. Every Server Action also checks the role and ownership again before changing data, for example:

```
const entry = await getLogbookEntryById(entryId);
if (!entry || entry.studentId !== user.id) {
  return { error: 'Entry not found.' };
}
```

Logbook Entries and Evidence Upload

createLogbookEntryAction validates the date, description and hours (more than 0 and at most 24) before saving. An evidence file is accepted only if its extension is PDF, PNG, JPG, JPEG or WEBP, and it is saved under a random UUID name so that a user can never choose or overwrite a file name on the server (OWASP Foundation, n.d.c). Files are served through /api/evidence/[fileName], which returns 401 to signed-out visitors and 403 to anyone other than the owning student, a coordinator or a lecturer (FR-13).

Mentor Verification Tokens

When a student submits an entry, the server creates a token from 24 random bytes (48 hexadecimal characters, 192 bits of randomness), which cannot practically be guessed, and sets it to expire after 7 days. The token and the new Pending verification status are saved in one transaction. When the mentor decides, the token is marked as used only if it is still unused, and the entry is updated in the same transaction:

```
const tokenUpdate = database
  .prepare(`UPDATE verification_tokens SET used_at = ?, decision = ?
    WHERE token = ? AND used_at IS NULL`)
  .run(decidedAt, decision, tokenValue);
if (tokenUpdate.changes === 0) {
  return false; // already used, e.g. a double click
}
```

Monitoring Dashboard

The dashboard calculates each student's status from their entries each time it is displayed. A student with an entry awaiting verification is shown as Awaiting verification; a student with no verified entry, or whose latest verification is more than 14 days old, is shown as Overdue; all other students are On track. Verified hours are the sum of the hours of Verified entries only.

Database Layer

db.ts opens the SQLite file, turns on foreign-key enforcement, applies schema.sql and loads seed.sql if the database is empty. It also provides an inTransaction helper that commits all statements together or rolls them all back. repository.ts contains every SQL query and returns plain TypeScript objects, so pages and actions never contain SQL.

Known Security Limitation

During the security review the team found that the session cookie stores the user's id without a signature. The cookie is HttpOnly, so it cannot be read by JavaScript on the page, but a user who edits the cookie in their own browser could change the id to impersonate another account. This defect is recorded in the Defect Log, and the planned fix is to sign the cookie (or use a random session token stored in the database) so that a modified cookie is rejected. Chapter 5 reports the retest.

4.8 Version Control

The repository on GitHub (https://github.com/Noobey32/TrackWIL) holds both the documentation and the source code. The capstone pack is organised into numbered folders from 00_Project_Administration to 12_Assessment, with the application in 05_Source_Code/trackwil, the design files and diagrams in 04_System_Design_and_Development, and each weekly submission in its own folder.

Branching and Pull Requests

The team agreed that each member creates a branch named after themselves and the task (name/task, for example moe/system-design), finishes the work there, and merges it into main through a pull request so that the others can review it. main is kept as the stable branch.

Table 18: Branches in the repository

Branch	Purpose	Merged into main
main	Stable, reviewed work	(base branch)
app-dev	First application development (Next.js scaffold and MVP)	Yes
moe/system-design	UML, ERD and architecture diagrams and the Chapter 4 draft	Yes
gedion/SRS_Documents_Chapter_3	Software Requirements Specification and Chapter 3	Pull request #2
Dylan/Revise-chapters-1-and-2	Revisions to Chapters 1 and 2 after Week 2 feedback	Yes
gedion/Final_Changes_Week_3_Submission	Week 3 submission compilation	Pull request #3
sihle-trackwil	Chapter 5, test plan and test cases	Pull request #4
gedion/sqlite-database	Replace JSON storage with the SQLite database	Pull request #5
gedion/docs-match-build	Update the documentation to match the final build	Pull request (Week 4)

Contribution by Member

Table 19: Commits and main contributions by member (at the time of writing)

Member	GitHub account	Commits	Main contributions shown in the history
Willem (Gedion) Basson	Cyber-4	20	Project proposal and topic acceptance; administration folder (00); repository structure; SRS and Chapter 3; Week 2 and Week 3 submission compilation; SQLite database migration; merging pull requests #2, #3 and #5
Moegammad Green	Noobey32 (repository owner)	16	Next.js scaffold and MVP; JSON storage and mentor verification; student sign-up validation; code documentation; draw.io diagrams; Chapter 4 draft; project charter and work plan
Dylan Shepard	FyreGH	12	Chapter 2 literature review and review matrix; reference list; formatting of Chapters 1–3; revised Chapters 1 and 2 table; Week 3 submission folder
Sihle Kayise	sihle588	4	Chapter 1 and Chapter 3 uploads; Chapter 5 draft, test plan and test cases; pull request #4

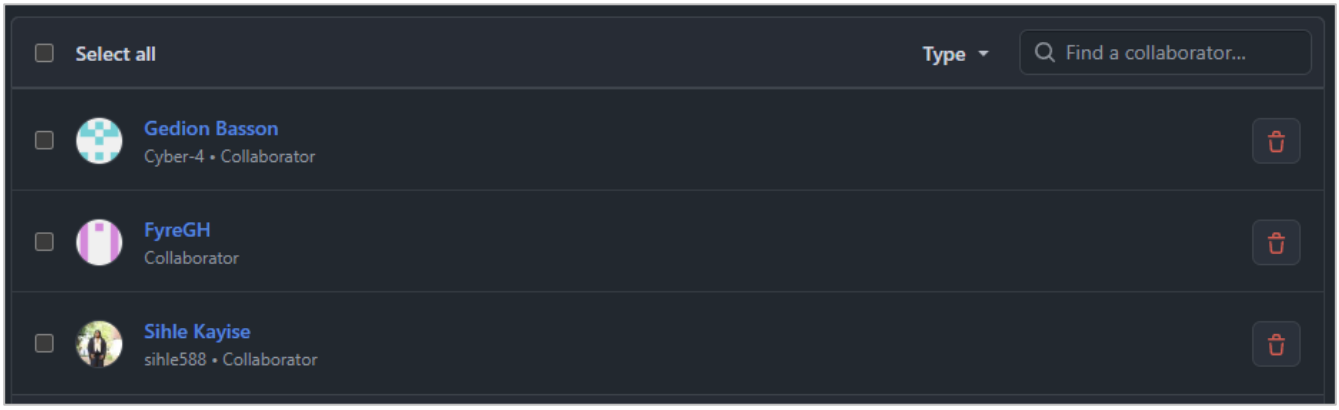


Figure 13: Collaborators on the GitHub repository

The commit history shows work on 7, 8, 9, 12, 14, 15 and 22 October 2026, spread across the project rather than in one final upload. The team recognises that the number of commits differs between members, and the Individual Contribution Forms explain each member's work in more detail, with references to specific commits and files.

4.9 Chapter Summary

This chapter presented TrackWIL's three-layer Next.js architecture, the use case, class, activity and sequence diagrams, the normalised SQLite database design and the user interface with its usability decisions. It described the development environment and the implementation of authentication, role-based access, evidence upload, one-time verification tokens, the monitoring dashboard and the database layer, and it recorded one known security limitation that will be fixed and retested. Finally, it summarised the repository structure, branches, pull requests and each member's contribution.

The core features implemented are: student sign-up and sign-in; logbook entries with optional evidence; verified hours against required hours; submission for verification with a 7-day one-time link; mentor approval or rejection without an account; and the coordinator and lecturer dashboard. Chapter 5 tests these features against the requirements.

CHAPTER 5 - TESTING, RESULTS & EVALUATION

5.1 Introduction

This chapter reports how TrackWIL was tested and evaluates the system against the requirements in Chapter 3. The purpose of testing was to show that every functional requirement works as specified, to measure the non-functional requirements wherever possible, and to find and fix defects before the final demonstration. The chapter describes the test strategy, summarises the test cases and their results, explains the defects that were found and corrected, presents the performance and usability evaluation, and evaluates each requirement. All test cases, defects and traceability links are recorded in the testing workbooks in the folder 06_Testing_Results_and_Evaluation.

5.2 Test Strategy

Testing followed the Test Plan and combined manual and automated testing at the system level.

- **Manual functional testing (TC-001 to TC-014).** Sihle Kayise tested the main user journeys in the browser on 22 October 2026: signing in and out as a student and as a coordinator, viewing and creating logbook entries, navigation, and invalid sign-in and empty-field validation.
- **Automated system testing (TC-110 to TC-144).** A test script (tests/system-tests.mjs, run with npm run test:system) sends the same requests as the browser to the running application and checks both the response and the database. It covers every functional requirement, the security rules, database constraints and response times, and can be repeated after every change.
- **Browser and device checks (TC-145 to TC-146).** The main screens were checked in Google Chrome 155 and Microsoft Edge 155 at a phone width of 375 pixels and at desktop width.
- **Code and data reviews (TC-147 to TC-148).** Type checking, linting, the README set-up steps and the stored data were reviewed.
- **Usability testing with users (TC-149).** Planned, but blocked until written campus permission is received.

Environment. Windows 11 Pro; Node.js 24.21; TrackWIL production build (npm run build, npm start) with the SQLite database reset to the synthetic seed data before each run; Chrome 155.0 and Edge 155.0. No real personal data was used.

Pass criteria. A test passes when the actual result matches the expected result recorded in the workbook. Defects are logged, fixed, and retested with the same test case, and the full automated suite is rerun after every fix (regression testing).

5.3 Test Cases and Results

Table 20: Summary of test results

Test group	Test cases	Requirements	Pass	Fail	Blocked
Manual functional tests (Sihle Kayise)	TC-001 to TC-014	FR-01, FR-04, FR-09 to FR-11, FR-14	14	0	0
Sign-in, sign-up and access control	TC-110 to TC-121 (incl. TC-120b)	FR-01 to FR-03, NFR-03	13	0	0

Test group	Test cases	Requirements	Pass	Fail	Blocked
Logbook entries and evidence	TC-122 to TC-126	FR-04, FR-05, FR-13	5	0	0
Verification links	TC-127 to TC-137	FR-06 to FR-08	11	0	0
Logbook, dashboard, timestamps and sign-out	TC-138 to TC-141	FR-09 to FR-12, FR-14	4	0	0
Security, integrity and performance	TC-142 to TC-144	NFR-02 to NFR-04	3	0	0
Browser, code and data checks	TC-145 to TC-148	NFR-05 to NFR-07	4	0	0
Usability with first-time students	TC-149	NFR-01	0	0	1
Total	55 test cases		54	0	1

All 55 test cases are recorded in the Test Cases and Results workbook with their preconditions, steps, test data, expected and actual results, tester, date and evidence. The automated run log (evidence/system_test_run_2026-10-23.txt) contains the actual output of every automated test. The final regression run on 23 October 2026 passed all 36 automated tests.

The results that matter most for the research aim are:

- **Account-free verification works end to end.** Submitting an entry produced a 48-character link that expires after exactly 7.00 days (TC-127). The mentor could open it without signing in (TC-130), approval set the entry to Verified with a timestamp (TC-131), and a rejected entry could be resubmitted with a new link (TC-132).
- **Links cannot be misused.** A used link (TC-133), an unknown link (TC-134) and an expired link (TC-135) were all refused without changing the entry, and two simultaneous approvals on the same link recorded only one decision (TC-136).
- **Access control holds.** Signed-out visitors, students and staff could reach only their own areas (TC-117 to TC-120b). Evidence files returned 200 to the owner and to a coordinator, 403 to another student, 401 to a signed-out visitor and 404 to a path-traversal attempt (TC-126).
- **Monitoring is accurate.** The dashboard showed Dylan as On track, Moe as Overdue and Sihle as Awaiting verification, matching the seed data (TC-139), and the logbook showed the correct verified hours (TC-138).

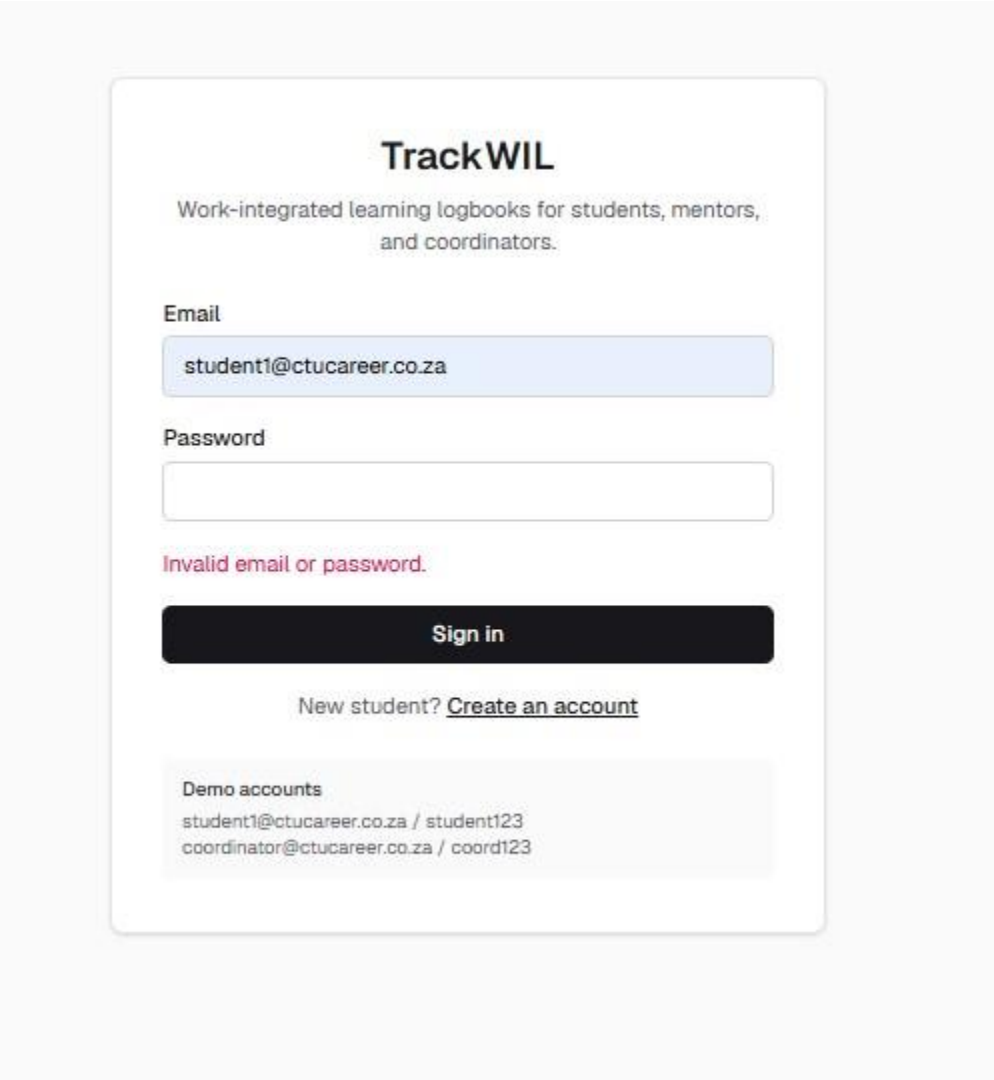


Figure 14: Manual test TC-011: an invalid sign-in is refused with a clear message (Sihle Kayise, 22 October 2026)

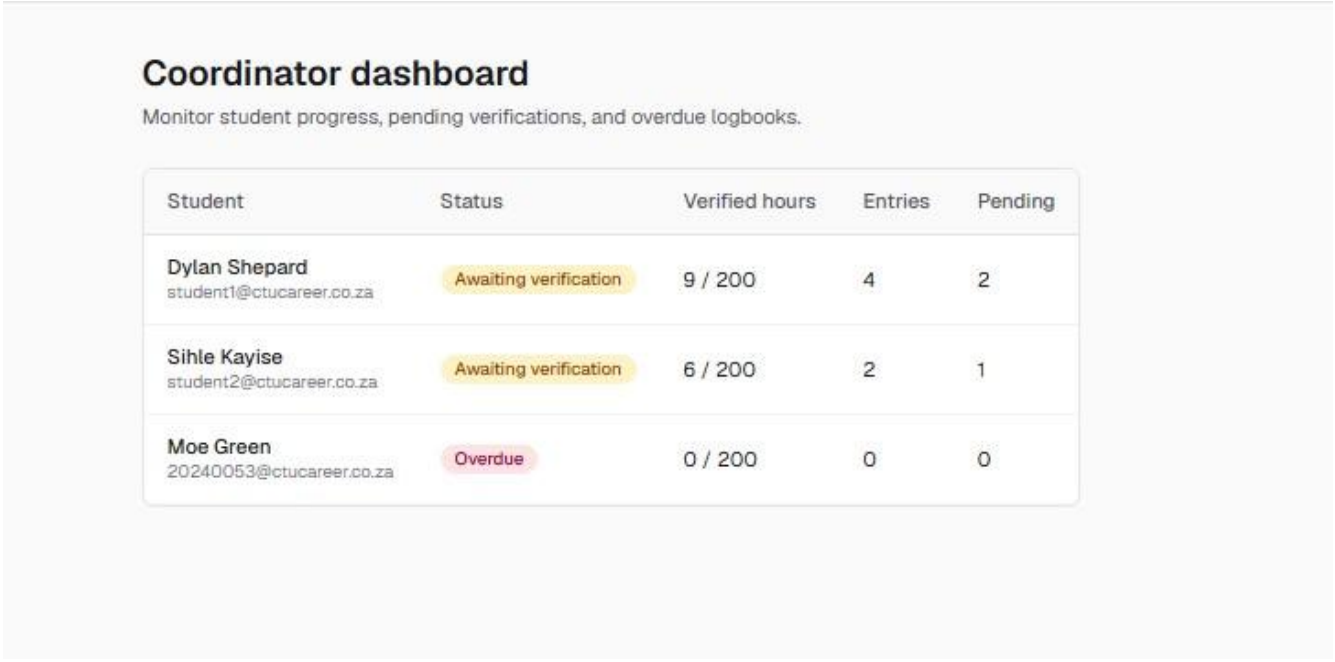


Figure 15: Manual test TC-009: coordinator dashboard during manual testing (Sihle Kayise, 22 October 2026)

5.4 Defects and Corrections

Five defects were logged during testing. The three that affected users were fixed, and each was retested with its own test case, followed by a full regression run. The two that remain open are low risk for the demonstration and have planned fixes.

Table 21: Defects found, fixes and retest results

ID	Defect	Severity	Fix	Retest
DEF-01	The session cookie held the user id without a signature. Setting the cookie to "coordinator-1" opened the coordinator dashboard without a password (reproduced on the earlier build: HTTP 200).	Critical	The cookie is now the user id plus an HMAC-SHA256 signature, checked in constant time; edited cookies are rejected.	Closed: TC-121 pass
DEF-02	If a mentor link expired before the mentor responded, the entry stayed Pending verification and could never be resubmitted.	Medium	An entry with no active link can be resubmitted; the entry page shows an expiry notice.	Closed: TC-137 pass
DEF-03	On a 375-pixel phone screen the logbook and dashboard tables widened the page to 580 pixels and cut off columns.	Medium	Full-width page container; tables scroll inside their box.	Closed: TC-145 pass (Chrome and Edge)
DEF-04	The build reports a known security vulnerability in Next.js 16.4.0.	High	Upgrade Next.js on a separate branch and rerun the test suite.	Open (planned before 29 October)
DEF-05	The sign-in page lists demo accounts but not the lecturer account.	Low	Add the lecturer account to the list.	Open

DEF-01 is the most important finding. It was found by reviewing the session code against the OWASP guidance used in Chapter 2, and it shows the value of testing security as well as functionality: every functional test passed before the fix, yet any user could have impersonated a coordinator. The defect was first reproduced on the previous build, then fixed and retested, and the automated suite now includes a test that forged and tampered cookies are rejected, so the problem cannot return unnoticed.

5.5 Performance / Usability Evaluation

Performance (NFR-02). Each main page was requested 10 times on the test computer with the demonstration data. Every response was far below the 2-second target.

Table 22: Measured response times (TC-144, 23 October 2026)

Page or action	Average (ms)	Maximum (ms)	Target
Sign-in page (/)	3	4	under 2 000 ms
Student logbook (/student)	5	9	under 2 000 ms
New entry form	4	5	under 2 000 ms
Coordinator dashboard	4	6	under 2 000 ms
Mentor verification page	4	5	under 2 000 ms
Create a logbook entry	20	20	under 2 000 ms

These times were measured on the same computer as the server, so they show that the application and database are fast. They do not include internet delays, which would add time in a hosted deployment.

Compatibility (NFR-06). All main screens worked in Chrome 155 and Edge 155. At 375 pixels the first check found DEF-03; after the fix every screen fits the phone width, and the mentor verification page, which mentors are most likely to open on a phone, needs no sideways scrolling at all.

Usability (NFR-01). The interface was designed for quick use (Chapter 4, Section 4.5): an entry needs four fields, statuses use coloured labels with text, and the mentor page has two buttons. However, the planned timed task with first-time students (TC-149) could not be run because written campus permission for testing with students has not yet been received. NFR-01 is therefore not yet proven by user data, and this is stated as a limitation.

5.6 Requirements Evaluation

Table 23: Evaluation against the requirements

Requirement	Result	Evidence
FR-01 to FR-03: sign-in, sign-up, role-based access	Met	TC-001, TC-008, TC-011, TC-013, TC-110 to TC-121
FR-04 to FR-05: logbook entries and evidence	Met	TC-005, TC-006, TC-012, TC-014, TC-122 to TC-125
FR-06 to FR-08: submission and one-time mentor links	Met	TC-127 to TC-137
FR-09 to FR-11: statuses, logbook and monitoring dashboard	Met	TC-002 to TC-004, TC-009, TC-138, TC-139

Requirement	Result	Evidence
FR-12 to FR-14: timestamps, protected evidence, sign-out	Met	TC-007, TC-010, TC-126, TC-140, TC-141
NFR-01 Usability	Partially met: designed for, not yet measured with users	TC-149 blocked
NFR-02 Performance	Met (local measurement)	TC-144
NFR-03 Security	Met after DEF-01 fix; dependency upgrade outstanding (DEF-04)	TC-121, TC-126, TC-127, TC-142
NFR-04 Reliability and integrity	Met	TC-136, TC-143
NFR-05 Maintainability	Met	TC-147
NFR-06 Compatibility	Met after DEF-03 fix	TC-145, TC-146
NFR-07 Privacy	Met	TC-148

All fourteen functional requirements are met, and six of the seven non-functional requirements are met. The Requirements Traceability Matrix links every requirement to its design component, implementation files and test cases. The requirements deferred in Chapter 3 (mentor accounts, written mentor feedback, mentor access to evidence, a separate lecturer dashboard, notifications and reports) were not part of version 1.0 and are carried into the recommendations in Chapter 6.

5.7 User / Stakeholder Feedback

No feedback has yet been collected from real students, mentors, coordinators or lecturers, because the Ethical Clearance Confirmation does not allow personal data to be collected before written campus permission is granted. The evaluation therefore relies on testing against the requirements and on the feedback that was available:

- **Supervisor feedback on the proposal (7 October 2026)** narrowed the scope to three core features, which allowed them to be completed and tested thoroughly.
- **Supervisor feedback on Week 2** asked the team to remove template formatting and fix scope inconsistencies; these changes were made in the revised chapters.
- **Internal review during testing** led to three improvements to the system: the signed session cookie (DEF-01), resubmission of expired links (DEF-02) and the mobile layout (DEF-03).

If permission is granted before the final submission, a short task-based test and questionnaire will be run with volunteer students and staff, and the results will be added to this section.

5.8 Chapter Summary

TrackWIL was tested with 55 test cases: 14 manual tests, 36 automated system tests and 5 browser, code and data checks. Fifty-four passed and one, the usability test with students, is blocked until ethical permission is granted. Testing found five defects. The three that affected users, including a critical session-cookie vulnerability, were fixed and retested, and the remaining two have planned fixes. All functional requirements and six of the seven non-functional requirements are met, and response times were far below the target. Within the limits of synthetic data and the absence of user feedback, the results show that TrackWIL supports the full WIL logbook workflow: recording daily work, account-free mentor verification and progress monitoring. Chapter 6 draws conclusions from these results.

CHAPTER 6 - CONCLUSION & RECOMMENDATIONS (OUTLINE)

6.1 Introduction

- Restate the problem: daily WIL logbooks are managed through paper, PDF and email, so records are lost, verification is slow and staff cannot see who is behind.
- Restate the aim: design, develop and evaluate TrackWIL, a web application for daily logbooks, account-free mentor verification and progress monitoring.

6.2 Summary of Findings

- The core workflow works end to end: entry, evidence, one-time mentor link, verification, hour totals and the monitoring dashboard (Chapter 5: 54 of 55 test cases passed; 1 blocked).
- Account-free verification through a single-use, 7-day link is practical and secure when the token is long and random, used only once and protected by a transaction.
- Security testing found a critical session-cookie defect that every functional test had missed; it was fixed and retested.
- Performance is far below the 2-second target (average 3 to 5 ms per page locally).
- Usability has not yet been measured with real users because ethical permission is pending.

6.3 Achievement of Objectives

Table 24: Achievement of objectives (to be expanded)

Objective	Status	Evidence
1. Analyse the current WIL documentation process and its challenges	Achieved	Chapters 1 and 2; requirements gathering in Chapter 3
2. Design a logbook system with account-free mentor verification	Achieved	Chapter 4: architecture, UML, ERD, UI design
3. Develop the student logbook, mentor page and shared dashboard	Achieved	Working application; Chapter 4.7; FR-01 to FR-14 met
4. Implement secure storage and status tracking	Achieved	SQLite schema with constraints and transactions; TC-136, TC-140, TC-143
5. Test and evaluate TrackWIL against its requirements	Largely achieved	Chapter 5; usability test with users outstanding

6.4 Answers to Research Questions

- **RQ1 (challenges of the current process):** lost or unreturned documents, delays waiting for signatures, no single view of progress, and extra work for external mentors (Chapters 1 and 2).
- **RQ2 (digital workflow with link-based verification):** a one-time link lets the mentor verify in two clicks without an account, and the status and time are recorded immediately (TC-127 to TC-137).
- **RQ3 (role-specific views):** students see their own logbook and hours, mentors see only the one entry in their link, and staff see every student's status (FR-03, FR-11).
- **RQ4 (central database):** one record per entry with its status and timestamps removes conflicting copies and makes outstanding work visible (Chapter 4.4; TC-139, TC-140).

- **RQ5 (extent of improvement):** all functional requirements and six of seven non-functional requirements are met; the efficiency gain for real users still needs to be measured with the timed usability test.

6.5 Limitations

- Synthetic data only; no feedback from real students, mentors or staff yet (ethical permission pending).
- Usability (NFR-01) not measured with users.
- Response times measured locally, not on a hosted server.
- Mentors cannot view evidence files or leave written feedback; no email notifications.
- A known framework vulnerability (Next.js 16.4.0, DEF-04) must be fixed by upgrading.
- A four-week build by a team of four limited the scope.

6.6 Recommendations

- Upgrade Next.js and keep dependencies up to date before any real use.
- Run the timed usability test and questionnaire once permission is granted, and use the results to refine the forms.
- Host the application with HTTPS, set TRACKWIL_SESSION_SECRET and back up the database daily before using real data.
- Agree data-retention and consent rules with the campus in line with POPIA.

6.7 Future Work

- Written mentor feedback on rejected entries, and secure mentor access to evidence through the link.
- Automatic email notifications for verification requests and expiring links.
- A separate lecturer view with drill-down into individual students' entries.
- Reports and data export for audits; an audit log of changes.
- Migration from SQLite to PostgreSQL for a multi-user hosted deployment.

6.8 Final Conclusion

- TrackWIL shows that a small, focused web application can replace the paper and email logbook process with a secure, traceable workflow.
- Its main contribution is account-free mentor verification combined with automatic progress monitoring.
- The project met its functional objectives; its effect on real users is the main question for the next stage.

References

Beck, K. et al. (2001) Manifesto for Agile Software Development. Available at: https://agilemanifesto.org/ (Accessed: 22 October 2026).

Hains-Wesson, R. et al. (2026) [TO COMPLETE: full title, journal, volume, issue, pages and DOI or URL of the systematic literature review on placement supervision cited in Section 2.2].

InPlace Software (n.d.) Work Integrated Learning Placement Software. Available at: https://inplacesoftware.com/higher-education/work-integrated-learning/ (Accessed: 9 October 2026).

Mahara Project (n.d.) About ePortfolios. Available at: https://mahara.org/view/view.php?id=3 (Accessed: 9 October 2026).

Mell, P. and Grance, T. (2011) The NIST Definition of Cloud Computing. NIST Special Publication 800-145. Gaithersburg, MD: National Institute of Standards and Technology. Available at: https://doi.org/10.6028/NIST.SP.800-145 (Accessed: 9 October 2026).

OpenJS Foundation (n.d.) Node.js documentation: SQLite. Available at: https://nodejs.org/api/sqlite.html (Accessed: 22 October 2026).

OWASP Foundation (n.d.a) Password Storage Cheat Sheet. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html (Accessed: 22 October 2026).

OWASP Foundation (n.d.b) Authorization Cheat Sheet. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html (Accessed: 9 October 2026).

OWASP Foundation (n.d.c) File Upload Cheat Sheet. Available at: https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html (Accessed: 9 October 2026).

PebblePad (2026) PebblePad for Assessors. Available at: https://helphub.pebblepad.com/support/solutions/articles/101000539607-pebblepad-for-assessors (Accessed: 9 October 2026).

Republic of South Africa (2013) Protection of Personal Information Act 4 of 2013. Pretoria: Government Printer.

Schuster, L. and Glavas, C. (2017) 'Exploring the dimensions of electronic work integrated learning (eWIL)'. Available at: https://www.sciencedirect.com/science/article/abs/pii/S1747938X17300155 (Accessed: 9 October 2026).

Schwaber, K. and Sutherland, J. (2020) The Scrum Guide. Available at: https://scrumguides.org/scrum-guide.html (Accessed: 22 October 2026).

Sommerville, I. (2016) Software Engineering. 10th edn. Harlow: Pearson.

Sonia Online (n.d.) [TO COMPLETE: page title and URL of the Sonia Online placement management website cited in Sections 2.2 and 2.3] (Accessed: 9 October 2026).

SQLite (n.d.) About SQLite. Available at: https://www.sqlite.org/about.html (Accessed: 22 October 2026).

Vercel (n.d.) Next.js Documentation. Available at: https://nextjs.org/docs (Accessed: 22 October 2026).

Appendices

Appendix A: Source Code and How to Run TrackWIL

The source code is in the folder 05_Source_Code/trackwil of the repository https://github.com/Noobey32/TrackWIL. To run it, install Node.js 22.13 or newer, then run npm install and npm run dev in that folder and open http://localhost:3000. The database is created and filled with synthetic data automatically.

Table 25: Demonstration accounts (synthetic data)

Role	Email	Password
Student	student1@ctucareer.co.za	student123
Student	student2@ctucareer.co.za	student123
Coordinator	coordinator@ctucareer.co.za	coord123
Lecturer	lecturer@ctucareer.co.za	lecturer123

Appendix B: Supporting Documents

Table 26: Supporting documents in the repository

Document	Location
Software Requirements Specification	03_Requirements_and_Methodology
Database Design document, UML and ER diagrams, UI screenshots	04_System_Design_and_Development
Test Plan, Test Cases and Results, Defect Log, Requirements Traceability Matrix, test evidence	06_Testing_Results_and_Evaluation
Project Charter, Sprint Backlog, Risk, Issue and Change Register, Weekly Progress Reports	09_Project_Management_Evidence
Demonstration script and slide outline	11_Presentation_and_Demonstration
Individual Contribution Forms	12_Assessment

Appendix C: Mobile Test Evidence

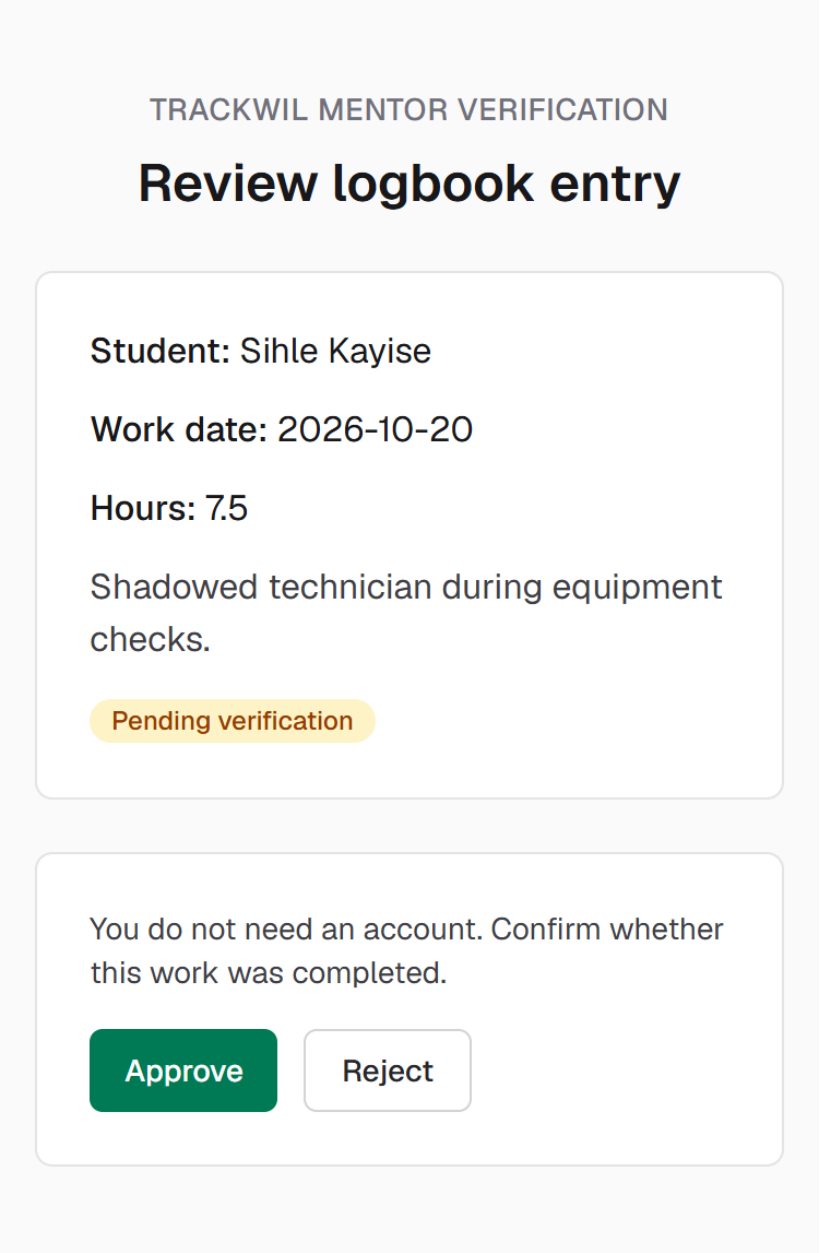


Figure 16: Mentor verification page at 375 px in Chrome after the DEF-03 fix (TC-145)

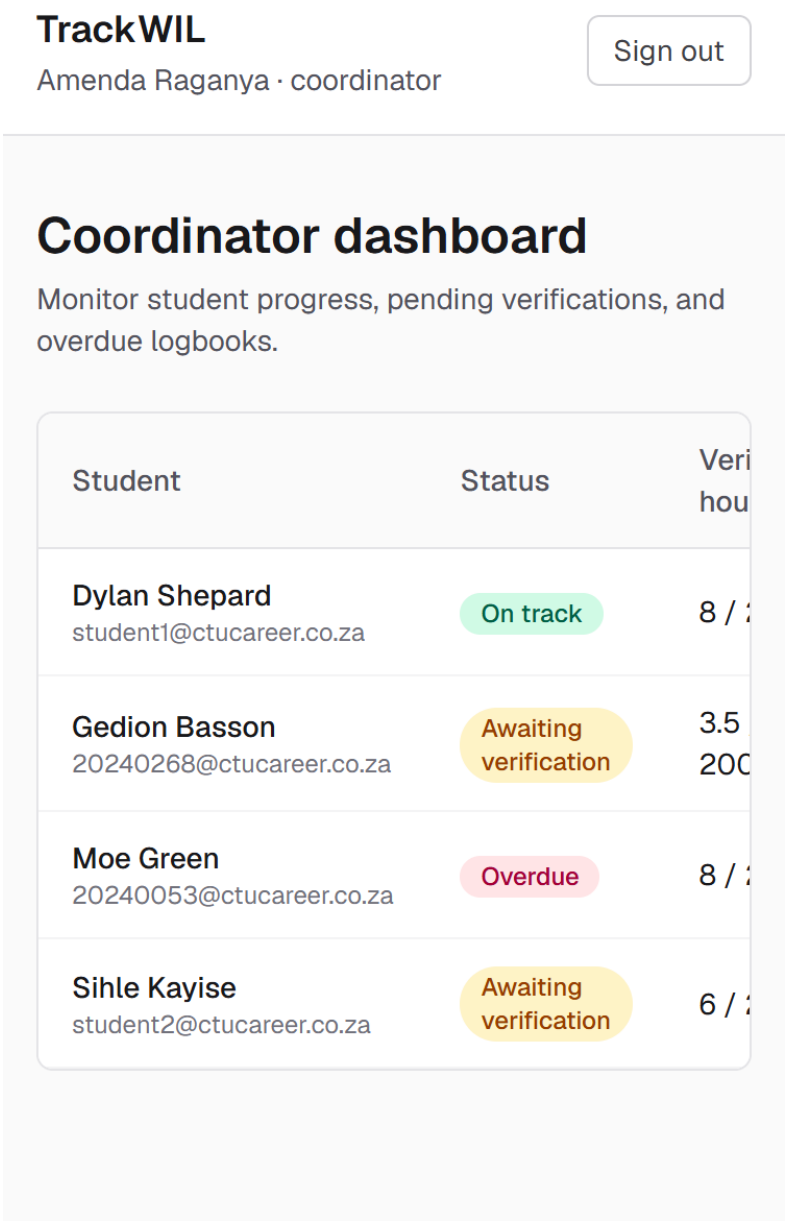


Figure 17: Coordinator dashboard at 375 px in Edge after the DEF-03 fix: the table scrolls inside its box (TC-145)